

Analyse de tavelures laser par apprentissage profond

Description et planification

Projet Intégrateur

Gabriel Genest

Sous la direction de :

Luc Lamontagne

Table des matières

Table des matières	ii
1 Description du problème	1
2 Revue de littérature	3
2.1 Approches classiques	3
2.2 Apprentissage automatique	4
3 Choix des modèles	5
3.1 Modèle initial scalaire	5
3.2 Modèle secondaire	6
3.3 Pipeline d'entraînement	6
4 Données du modèle	9
4.1 Générations de base	9
4.2 Générations avancées : à utiliser	10
5 Évaluation du modèle	15
6 Tâches à accomplir	16
6.1 Génération des données	16
6.2 Développement du pipeline d'entraînement : DataLoader et Dataset	16
6.3 Développement du pipeline d'entraînement : Modèle	17
6.4 Décision du modèle	17
6.5 Exploration complémentaire (si le temps le permet)	17
7 Expérimentations initiales	18
7.1 Sur-apprentissage	18
8 Résultats initiaux	25
8.1 Jeux de données	25
8.2 Premier réseau	26
8.3 Deuxième réseau	27
8.4 Troisième réseau	27
8.5 Quatrième réseau	28
8.6 Cinquième réseau	29
8.7 Autres métriques d'analyse	30
8.8 Analyses supplémentaires	40

9	Seconde partie du projet	42
9.1	Optimisation	42
9.2	Varier les conditions	42
9.3	Sélection d'un modèle scalaire final	43
9.4	Modèle cartographique : un aperçu	43
	Bibliographie	44

Chapitre 1

Description du problème

Le projet présenté dans ce document est centré sur une problématique touchant l'imagerie et la microscopie. Ces domaines sont au centre de la médecine, de la biophotonique et de plusieurs procédés industriels. Alors que plusieurs cherchent à améliorer la résolution en dépassant le critère de Rayleigh, il existe une modalité d'imagerie simple et naturelle qui a gagné en popularité au cours des dernières années, malgré son émergence au cours des années 1980. Il s'agit de l'imagerie à l'aide des tavelures laser, ou encore *laser speckle imaging* en anglais. Les tavelures laser apparaissent lorsqu'un faisceau de lumière cohérente frappe une surface ayant des aspérités microscopiques de sorte que la lumière est diffusée dans des directions quelconques qu'on simplifie comme aléatoires. Cette diffusion cause différents délais pour les ondes réfléchies, créant des interférences destructives ou constructives lors de la détection de la lumière. La base mathématique des tavelures est bien connue du monde de l'optique statistique (voir les références [5 ; 4 ; 3] pour plus d'informations), mais une utilisation efficace et rapide des tavelures pour l'imagerie reste une étape cruciale. L'analyse en temps réel des tavelures permet de suivre l'évolution de matériaux ou de phénomènes biologiques. Pour cela, l'apprentissage profond est particulièrement adapté pour automatiser et potentiellement accélérer l'analyse de tavelures, que ce soit dans le domaine spatial ou temporel.

Dans un premier temps, il existe des méthodes d'analyses statistique classiques, mais étant donné la nature stochastique des tavelures, ces méthodes varient et peuvent induire un biais. Ces analyses sont souvent assez lourdes mathématiquement et computationnellement, demandant par exemple plusieurs images dans un film ou encore des propriétés stochastiques bien spécifiques, comme l'ergodicité. Il est possible de modifier ces méthodes si certaines conditions ne sont pas respectées, mais savoir si elles le sont est souvent une tâche supplémentaire apportant son lot de défis. Une approche par apprentissage profond pourrait se spécialiser selon les besoins sans avoir à changer la structure. Si les tavelures sont stationnaires dans l'espace ou non, le réseau devrait être capable de distinguer sans l'intervention humaine lors de l'analyse.

Dans un second temps, plusieurs analyses sont spécifiques à une problématique bien particulière. Cela multiplie les modèles et algorithmes, menant à une recherche difficile pour trouver le bon moyen. Par exemple, si on est intéressé à calculer la vitesse de déplacement des éléments diffuseurs, on doit avoir un modèle qui la calcule en fonction des données statistiques contenues dans les tavelures. Si on est plutôt intéressé à calculer la rigidité du matériau, on doit utiliser un modèle différent faisant le pont entre les statistiques des tavelures et le module de viscoélasticité, par exemple. Dans ces deux cas, les modèles risquent d'être assez différents ou encore ayant une construction difficile à implémenter. Utiliser une approche par apprentissage profond pourrait, selon l'entraînement, construire son propre modèle interne et apprendre comment extraire ces informations sans interventions humaine lors de l'analyse.

Comme le phénomène des tavelures laser est autant spatial que temporel, un réseau alliant le contenu spatial et le contenu temporel est considéré. Le réseau aura comme entrée une séquence d'images de tavelures dans le temps et devra sortir l'information pertinente voulue. Dans le cadre de ce projet, cette information est le temps caractéristique de corrélation. La tête du réseau pourrait être modifiée selon l'information à extraire, par exemple la vitesse des diffuseurs. Une architecture mixte composée d'un réseau à convolution (CNN) et d'un réseau récurrent (RNN) sera utilisé, permettant un travail indépendant dans l'espace avec le CNN et l'information du CNN sera par la suite fournie au RNN pour l'aspect temporel. Ainsi, le réseau devrait être capable de cibler les informations pertinentes dans la corrélation spatiale et celle temporelle pour développer son modèle interne.

Chapitre 2

Revue de littérature

2.1 Approches classiques

Plusieurs articles présentent des limitations de modèles qui seront à considérer durant le développement du projet :

- Briers et al. [1] expliquent quelques problèmes des modèles fixes de l’analyse des tavelures. Par exemple, certains modèles requièrent la connaissance a priori de la distribution de la vitesse des diffuseurs, ce qui n’est généralement pas possible, mais où on peut faire certaines hypothèses. La présence de diffuseurs statiques et dynamiques change aussi l’interprétabilité, car les statistiques changent spatialement, brisant la stationnarité et l’ergodicité parfois requise.
- Liu et al. [12] expliquent la multiplication des hypothèses et des modèles. Selon les hypothèses, les modèles changent et deviennent plutôt complexes. Dans cet article, on calcule le *blood flow index*, soit un indicateur de rapidité de déplacement du sang. Pour y arriver, il faut calculer le contraste et celui-ci est inversement proportionnel au *blood flow index*. Les modèles présentés comportent des paramètres expérimentaux importants, dont le paramètre β qu’on rencontre souvent dans la littérature des tavelures et qui est difficile à estimer.
- Zheng et Mertz [15 ; 14] expliquent, dans un premier temps, comment calculer directement le temps de corrélation d’une manière se rapprochant d’une approche sans modèle. On se base sur la théorie des tavelures pleinement développées et on intègre numériquement le contraste. Il y a toutefois les enjeux d’ergodicité : si les tavelures ne sont pas ergodiques, des corrections doivent être apportées, ajoutant une couche d’a priori. De plus, leur algorithme reste relativement lourd, utilisant une approche pixel par pixel. Dans un second temps, ils présentent une manière de corriger le biais induit lors du calcul local du contraste spatial. On ajoute encore une couche de complexité et le besoin de savoir si les tavelures sont ergodiques.

Ces articles présentent des approches classiques, utilisant des méthodes statistiques.

2.2 Apprentissage automatique

Au cours des dernières années, les chercheurs ont utilisé de plus en plus l'apprentissage machine pour analyser les tavelures :

- Fredriksson et al. [2 ; 7] expliquent l'utilisation d'un réseau de neurones artificiels dans le contexte de l'imagerie de tavelures à différents temps d'acquisition. Ils y utilisent un réseau de type perceptron, pleinement connecté avec une seule couche cachée. Le modèle est fourni d'informations extraites d'images de tavelures. Différents réseaux sont initialisés aléatoirement et le meilleur est retenu pour le test. L'utilisation d'une fonction d'activation tanh est expliquée par le fait que les réseaux l'ayant sont meilleurs que ceux ayant d'autres fonctions d'activation. Ils arrivent à la conclusion que l'apprentissage machine permet aux tavelures laser d'être utilisées et améliorer l'interprétation des images.
- Kaler et al. [9] utilisent différentes approches d'apprentissage automatique pour la détection de graines infectées. Dans un premier temps, ils utilisent des approches plus classiques, comme des SVMs ou des KNNs, mais ensuite combinent des CNNs avec des LSTMs pour une analyse spatio-temporelle. Ils arrivent à la conclusion qu'un modèle à LSTM convolutionnel est meilleur que les autres en général. Cet article est particulièrement intéressant, car il compare plusieurs architectures et une méthode d'analyse robuste, soumettant les architectures à des données originales, sans bruit, puis ajoutant du bruit. De plus, parmi les autres articles, ils sont les seuls à considérer l'aspect spatial et temporel en même temps, profitant des CNNs et des RNNs. En revanche, il s'agit ici d'un problème de classification.
- Shang et al. [13] utilisent un CNN tridimensionnel pour analyser le flot sanguin à partir d'images de tavelures laser. Ils arrivent à des précisions supérieures à 90%, mais font appel à une approche de classification, discrétisant le flot sanguin en plusieurs intervalles. Cette approche peut être avantageuse dans la prédiction, mais on perd de l'information qui est parfois très importante. Il faut aussi bien découper l'espace des résultats pour ne pas avoir un biais trop élevé pour certains intervalles. L'approche de CNN 3D reste toutefois très intéressante.

Chapitre 3

Choix des modèles

La programmation des modèles se fera à l'aide du module *PyTorch* du langage de programmation *Python*. Le code utilisé dans le cadre de ce projet est disponible sur le GitHub public <https://github.com/GabGabG/ProjetIntegrateur-DESS>, qui est en constante évolution. À noter que les données ne se trouvent pas dans le répertoire GitHub, car elles sont trop volumineuses. Elles se trouvent sur le serveur Valéria (plateforme ICE) de l'auteur et disponibles si besoin.

3.1 Modèle initial scalaire

Le modèle principal sélectionné est un mixte entre un réseau convolutionnel et un réseau récurrent, similaire à un modèle présenté dans [9]. Ce choix s'explique par la nature des tavelures et des statistiques importantes. Dans un premier temps, l'aspect spatial des tavelures est important à considérer. Comme il s'agit d'un motif d'interférence et d'imagerie, il est possible d'avoir des sections d'images qui sont sans information pertinente. Par exemple, des minimums sombres ou une absence d'illumination laissant des pixels sans intensité. L'avantage du CNN est aussi d'effectuer une sélection des régions importantes, diminuant la dimensionnalité. Il est important de noter que le pooling ne sera pas utilisé entre les couches de CNN, car cette étape diminue la qualité spatiale de l'information, parfois cachée. Il faut aussi mentionner que le réseau CNN ne devra pas être très profond étant donné que seules des statistiques de premier et de second ordre sont pertinentes. Utiliser un réseau trop profond pourrait mener à l'extraction d'information non pertinente rendant l'apprentissage plus difficile. À la fin de l'étape de convolution, un mécanisme d'attention spatiale sera utilisé pour sélectionner les combinaisons de caractéristiques pertinentes. Ce mécanisme est composé d'un petit perceptron multicouches (MLP) permettant de réduire la dimensionnalité des caractéristiques, forçant le réseau à apprendre comment réduire. Comment les caractéristiques sont spatiales, le MLP sert d'attention spatiale. Cette méthode est inspirée par la structure *Network In Network*[11] et par les architectures *Squeeze-and-Excitation Networks*[6].

Dans un second temps, un RNN de type GRU sera utilisé pour extraire l'information temporelle à partir de l'attention spatiale. Pour y arriver, un pooling sera appliqué afin de vectoriser les caractéristiques. Par la suite un réseau de type GRU sera entraîné sur une séquence temporelle de tavelures pour extraire l'information scalaire du temps caractéristique. Le choix d'un GRU repose principalement sur sa simplicité contrairement à un LSTM. Il sera toujours possible de tester un LSTM pour vérifier les performances. L'avantage d'un réseau récurrent est dans sa capacité à se souvenir ou oublier l'information passée d'une séquence, ce qui est important dans le cas des tavelures, car pour le temps caractéristique de corrélation, l'information réside dans toute la séquence temporelle et permettra d'estimer un scalaire en prédiction. Le nombre de couches sera petit pour éviter le surapprentissage : débiter avec une seule couche et augmenter au besoin. En plus des caractéristiques extraites du CNN, le temps d'intégration est donné au RNN. Cette quantité est normalement connue lors des expériences et peut servir à déterminer le temps caractéristique. On trouve l'implémentation de ce modèle dual CNN + RNN dans le fichier `projetIntegrateur/src/models/base_model.py` du répertoire GitHub.

3.2 Modèle secondaire

Si le temps le permet ou si le modèle principal s'avère problématique, le modèle secondaire est à étudier. Il s'agit d'un réseau récurrent convolutionnel, ConvRNN, de type ConvLSTM, comme utilisé dans [9]. Un avantage de cette approche est la possibilité d'extraire une carte locale du temps caractéristique de manière plus naturelle que le modèle précédent. Cependant, l'interprétation du modèle reste un peu plus difficile, demandant plus de paramètres, et pourrait être plus long à l'entraînement. De plus, *PyTorch* ne permet pas explicitement de créer un réseau ayant des couches ConvRRN, que ce soit ConvLSTM ou autre. Il existe toutefois une page GitHub où ConvLSTM est construit sur les bases de *PyTorch*, voir le lien https://github.com/ndrplz/ConvLSTM_pytorch, ayant plus de 2000 étoiles et 444 *forks*.

3.3 Pipeline d'entraînement

Les données utilisées pour l'entraînement sont présentées au chapitre 4. Les données générées peuvent être sous plusieurs formats, mais on va se contenter à les laisser en *ndarray* de *NumPy*, facilement transférable à *PyTorch*. Si besoin, on peut sauvegarder les données au format *.numpy* et les stocker sur un disque. *PyTorch* permet d'utiliser des utilitaires de création de datasets d'entraînement, de validation et de tests, ce qui sera utilisé pour s'assurer d'avoir un entraînement en *batch* avec un certain aléatoire pour diversifier le contenu de chaque *batch*.

Le modèle initial est schématiquement présenté à la figure 3.2, avec les hyperparamètres ini-

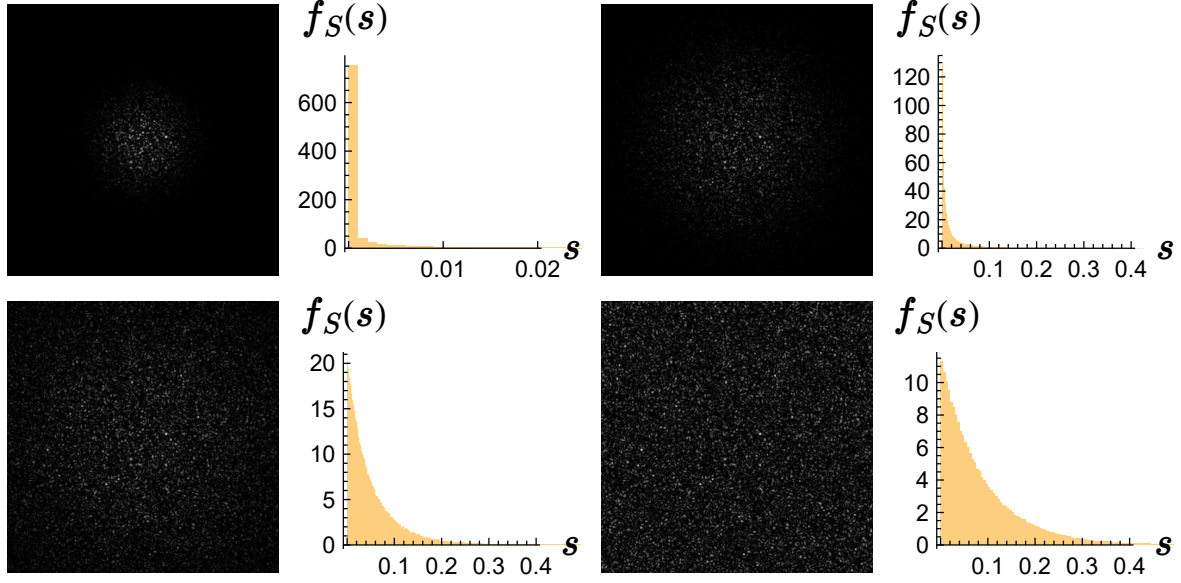


FIGURE 3.1 – Simulations de tavelures pleinement développées où un masque gaussien est appliqué sur l'intensité résultante, montrant l'effet d'un faisceau gaussien sur la tache laser. On montre que plus la tache laser est petite sur l'image, plus l'intensité est biaisée vers 0. De telles images doivent être normalisée judicieusement. Image tirée de [3].

tiaux. Ceux-ci peuvent changer durant le développement du projet. Pour l'entraînement, plusieurs modalités de simulations seront présentées aléatoirement à chaque batch. Les modalités pour chaque séquence seront décrites dans un fichier csv de métadonnées. Par modalité, on fait référence aux paramètres physiques de l'imagerie, comme le temps caractéristique, le temps d'intégration et la fonction de corrélation. Comme ces paramètres jouent un rôle important dans l'entraînement, plusieurs instances de chaque combinaison seront requises. Le DataLoader permet de bien distribuer aléatoirement les diverses simulations pour que le réseau ait une vue d'ensemble et non seulement quelques exemples ayant les mêmes propriétés. On trouve dans le fichier `projetIntegrateur/src/models/dataset.py` du répertoire GitHub une implémentation d'un Dataset propre au projet présent, avec notamment une lecture de fichiers d'images selon un fichier maître de métadonnées csv. Cette classe fonctionne directement avec un DataLoader de *PyTorch* pour effectuer un entraînement en mini-lots.

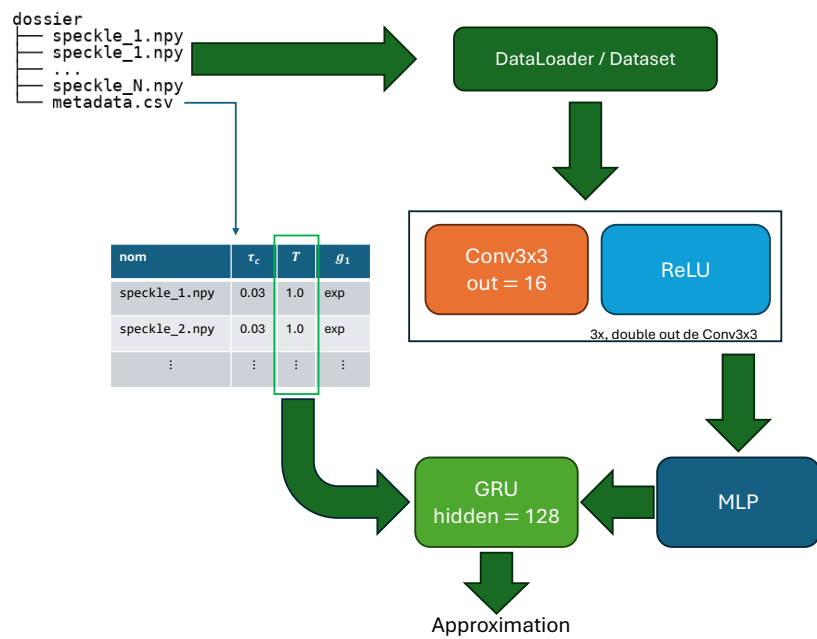


FIGURE 3.2 – Schéma montrant le pipeline du réseau utilisé pour le projet. Les hyperparamètres peuvent changer au cours du développement.

Chapitre 4

Données du modèle

4.1 Générations de base

Les données d'entraînement du modèle seront générées numériquement selon un algorithme précis. De cette manière, on peut modifier les paramètres statistiques des tavelures comme on veut. De plus, une grande quantité de données peuvent être générées dans une fraction du temps que demande un montage expérimental. L'algorithme est basé principalement sur [8] où on présente une façon de simuler l'intégration temporelle des tavelures. Toutefois, cet algorithme ne simule pas de séries temporelles. Initialement, on génère les phaseurs selon l'Algorithme 1.

Algorithme 1 Simulation de tavelures pleinement développées intégrées temporellement

Entrées : $N \in \mathbb{N}^*$, $R \in \mathbb{R}_+^*$, $N_T \in \mathbb{N}^*$, $\Delta T \in \mathbb{R}_+^*$, $g : \mathbb{R} \rightarrow \mathbb{R}$

Sorties : I avec $I \in \mathbb{R}_+^{N \times N}$

Générer $\Theta \in [-\pi, \pi]^{N \times N}$ avec $\Theta_{ik} \sim \mathcal{U}(-\pi, \pi)$ indépendants

Définir $\Sigma \in \{0, 1\}^{N \times N}$ comme un disque centré de rayon R

$T \leftarrow [0, \Delta T, 2\Delta T, \dots, (N_T - 1)\Delta T]$

$I \leftarrow \mathbf{0}_{N \times N}$

$G \leftarrow \mathbf{0}_{N_T \times N_T}$

Pour $i = 1$ **à** N_T **faire**

Pour $k = 1$ **à** N_T **faire**

$G_{ik} \leftarrow g(|T_i - T_j|)$

Fin Pour

Fin Pour

$\lambda \leftarrow \text{eigvals}(G)/N_T$

▷ eigvals calcule les valeurs propres

Pour $n = 1$ **à** N_T **faire**

$A \leftarrow \exp\{j\Theta\}$

▷ Exponentielle terme à terme

$A \leftarrow \mathcal{F}_{2D}(A)\sqrt{\lambda_n}$

▷ Transformée de Fourier 2D

$A \leftarrow \Sigma \odot A$

$A \leftarrow \mathcal{F}_{2D}^{-1}(A)$

▷ Transformée de Fourier inverse 2D

$A_{\mathbb{R}} \leftarrow |A|$

▷ Module terme à terme

$I \leftarrow I + A_{\mathbb{R}}^2$

▷ Carré terme à terme

Fin Pour

Pour une série temporelle, sans intégration temporelle, on doit modifier la création des amplitudes complexes pour ajouter une dimension temporelle ayant une certaine corrélation, comme montré à l'Algorithme 2. La corrélation provenant de la matrice de corrélation C est conservée dans le champ réel $A_{\mathbb{R}}$ et la corrélation de l'intensité I en est simplement le carré.

4.2 Générations avancées : à utiliser

Il est possible de mettre en commun l'Algorithme 1 et l'Algorithme 2 : on génère les amplitudes selon l'Algorithme 2, permettant ainsi une corrélation temporelle spécifique, puis ensuite on se base sur l'Algorithme 1. On obtient l'Algorithme 3. Un avantage de cet algorithme est qu'il génère des séries temporelles stationnaires grâce à la création des séries temporelles multinormales, contrairement à d'autres algorithmes trouvés dans la littérature. On peut alors décider de la corrélation temporelle, tant qu'elle donne une matrice de corrélation valide, c'est-à-dire positive (semi) définie. On peut aussi moduler le temps d'intégration simulée, donnant des images plus ou moins floues. Un désavantage de cette approche est sa complexité en mémoire. En effet, on doit générer $N_T \times P \times N \times N$ amplitudes complexes, même si au final on perd la première dimension à l'aide d'une réduction. L'interdépendance des P champs $N \times N$ fait aussi en sorte qu'on ne peut pas les générer *on the go*, on doit les créer toutes en même temps pour que la dépendance temporelle soit cohérente. Il est toutefois possible de vectoriser l'algorithme pour tirer avantage des architectures GPU. La première étape de

Algorithme 2 Simulation temporelle de tavelures pleinement développées

Entrées : $N \in \mathbb{N}^*$, $R \in \mathbb{R}_+^*$, $P \in \mathbb{N}^*$, $C \in \mathbb{R}^{P \times P}$

Sorties : $(A_{\mathbb{R}}, I)$ avec $A_{\mathbb{R}} \in \mathbb{R}^{P \times N \times N}$ et $I \in \mathbb{R}_+^{P \times N \times N}$

Générer $\mathcal{R} \in \mathbb{R}^{P \times N \times N}$ avec $\mathcal{R}_{pik} \sim \mathcal{N}(0, \frac{1}{2})$ et $\rho(\mathcal{R}_{pik}, \mathcal{R}_{p'i'k'}) = \begin{cases} C_{pp'} & \text{si } (i, k) = (i', k') \\ 0 & \text{sinon} \end{cases}$

Générer $\mathcal{I} \in \mathbb{R}^{P \times N \times N}$ avec $\mathcal{I}_{pik} \sim \mathcal{N}(0, \frac{1}{2})$ et $\rho(\mathcal{I}_{pik}, \mathcal{I}_{p'i'k'}) = \begin{cases} C_{pp'} & \text{si } (i, k) = (i', k') \\ 0 & \text{sinon} \end{cases}$

Définir $\Sigma \in \{0, 1\}^{N \times N}$ comme un disque centré de rayon R

$A \leftarrow \mathbf{0}_{P \times N \times N}$

Pour $p = 1$ **à** P **faire**

$A_p \leftarrow \mathcal{R}_p + j\mathcal{I}_p$

$A_p \leftarrow \mathcal{F}_{2D}(A_p)$

$A_p \leftarrow \Sigma \odot A_p$

$A_p \leftarrow \mathcal{F}_{2D}^{-1}(A_p)$

Fin Pour

$A_{\mathbb{R}} \leftarrow |A|$

$I \leftarrow A_{\mathbb{R}}^2$

vectorisation se trouve dans la boucle interne finale sur P , ainsi les opérations internes peuvent se faire en bloc. Une deuxième étape est d'aussi vectoriser la boucle externe finale sur N_T , mais on multiplie la mémoire requise en une passe. Soit on a beaucoup de VRAM à consacrer, soit on doit avoir de petites simulations. En se limitant par contre à la boucle interne, des GPUs de moyenne gamme peuvent très bien gérer des simulations intéressantes. On peut aussi s'assurer de libérer l'espace dès que possible ou encore de restreindre la précision des flottants, par exemple à 32 bits. De cette manière, les amplitudes complexes sont sur 64 bits, ce qui est suffisant en général.

Algorithme 3 Simulation temporelle de tavelures pleinement développées intégrées temporellement

Entrées : $N \in \mathbb{N}^*$, $R \in \mathbb{R}_+^*$, $N_T \in \mathbb{N}^*$, $\Delta T \in \mathbb{R}_+^*$, $g : \mathbb{R} \rightarrow \mathbb{R}$, $P \in \mathbb{N}^*$

Sorties : I avec $I \in \mathbb{R}_+^{P \times N \times N}$

Définir $\Sigma \in \{0, 1\}^{N \times N}$ comme un disque centré de rayon R

$T \leftarrow [0, \Delta T, 2\Delta T, \dots, (N_T - 1)\Delta T]$

$I \leftarrow \mathbf{0}_{P \times N \times N}$

$G \leftarrow \mathbf{0}_{N_T \times N_T}$

$c \leftarrow \mathbf{0}_P$

Pour $p = 0$ à $P - 1$ **faire**

$c_p \leftarrow g(p \cdot (N_T - 1)\Delta T)$

Fin Pour

$C \leftarrow \text{Toeplitz}(c)$

▷ Construit une matrice Toeplitz avec première ligne c

Pour $i = 1$ à N_T **faire**

Pour $k = 1$ à N_T **faire**

$G_{ik} \leftarrow g(|T_i - T_j|)$

Fin Pour

Fin Pour

$\lambda \leftarrow \text{eigvals}(G)/N_T$

Pour $n = 1$ à N_T **faire**

Générer $\mathcal{R} \in \mathbb{R}^{P \times N \times N}$, $\mathcal{R}_{pik} \sim \mathcal{N}(0, \frac{1}{2})$ et $\rho(\mathcal{R}_{pik}, \mathcal{R}_{p'i'k'}) = \begin{cases} C_{pp'} & \text{si } (i, k) = (i', k') \\ 0 & \text{sinon} \end{cases}$

Générer $\mathcal{I} \in \mathbb{R}^{P \times N \times N}$, $\mathcal{I}_{pik} \sim \mathcal{N}(0, \frac{1}{2})$ et $\rho(\mathcal{I}_{pik}, \mathcal{I}_{p'i'k'}) = \begin{cases} C_{pp'} & \text{si } (i, k) = (i', k') \\ 0 & \text{sinon} \end{cases}$

Pour $p = 1$ à P **faire**

$A_p \leftarrow \mathcal{R}_p + j\mathcal{I}_p$

$A_p \leftarrow \mathcal{F}_{2D}(A_p)\sqrt{\lambda_n}$

$A_p \leftarrow \Sigma \odot A_p$

$A_p \leftarrow \mathcal{F}_{2D}^{-1}(A_p)$

$A_{\mathbb{R}} \leftarrow |A_p|$

$I_p \leftarrow I_p + A_{\mathbb{R}}^2$

Fin Pour

Fin Pour

La génération des données se fera à l’aide du langage de programmation *Python*, en utilisant les puissantes librairies *NumPy* et *SciPy* principalement. De plus, pour l’accélération GPU, la librairie *CuPy* sera aussi importante, étant essentiellement un portage de *NumPy* (et d’un peu de *SciPy*) sur les cartes graphiques compatibles CUDA ¹. Pour plus d’information sur le module, visiter le site web <https://cupy.dev/>.

Si le temps le permet, il serait aussi intéressant de tester différentes variantes, notamment l’ajout de bruit, la non-uniformité de la tache laser et la variation spatiale du temps caractéristique de corrélation. Les deux premiers cas sont relativement triviaux, tandis que le dernier requiert une refonte de l’algorithme à l’étape de création des amplitudes complexes et lors de la pondération par les valeurs propres. Pour l’instant, seules les taches uniformes sans bruit et de temps de corrélation stationnaire dans l’espace sont considérées. La figure 4.1 montre trois simulations d’une même séquence où le temps d’intégration est de 0.03 seconde et le temps caractéristique est de 1 seconde. La corrélation de la séquence au complet est présentée à la figure 4.3a. À la figure 4.2, on montre trois simulations aussi d’une même séquence, avec les mêmes paramètres, sauf la corrélation qui est maintenant gaussienne au lieu d’exponentielle. La corrélation de la séquence au complet est présentée à la figure 4.3b. Ce genre d’images seront générées pour différents paramètres et serviront à l’entraînement, la validation et le test. Le code pour générer de telles simulations est présent dans le fichier `projetIntegrateur/src/simulations/time_integrated_sims.py`. On peut ajuster les paramètres d’entrées de l’Algorithme 3 et facilement générer un nombre voulu de séquences temporelles. À noter que ce code requiert l’installation du module *CuPy* et ne fonctionne pas autrement.

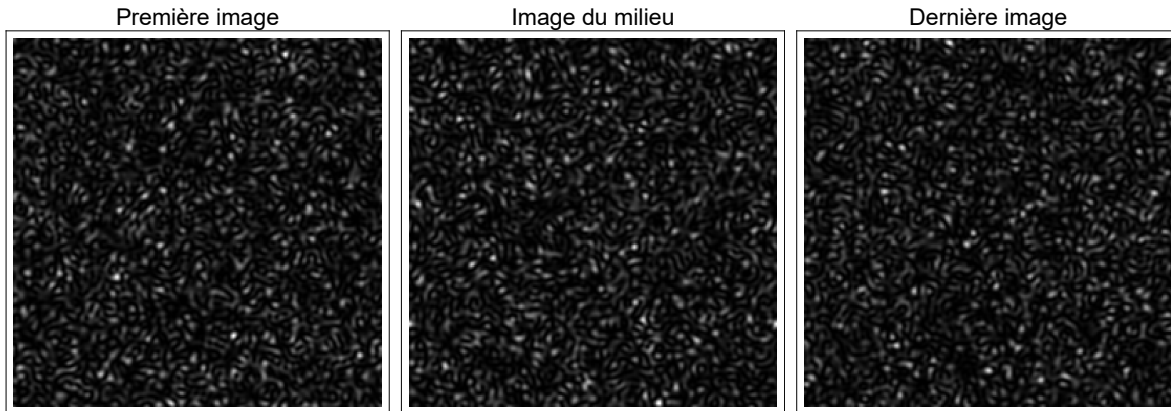


FIGURE 4.1 – Exemple d’une modalité de simulation de tavelures. Ici, la corrélation temporelle est exponentielle.

1. Aussi disponible pour les architectures ROCm, mais à l’état expérimental.

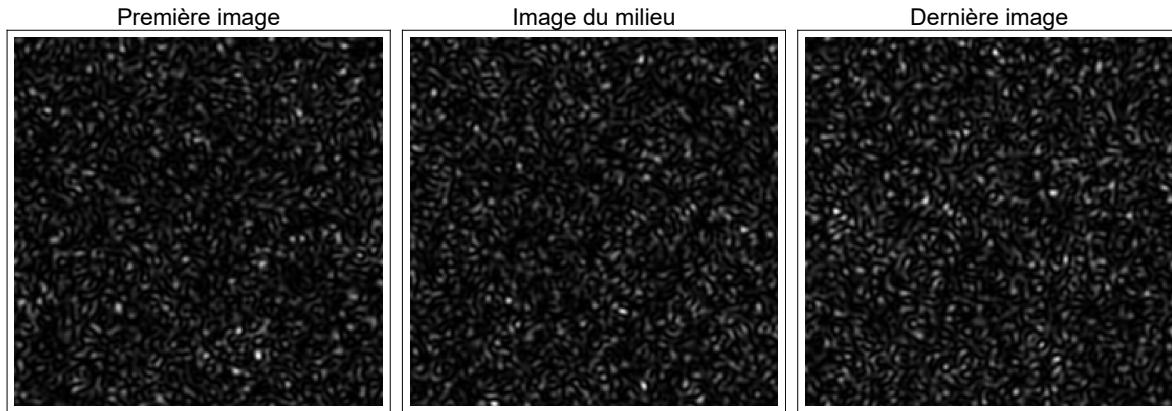
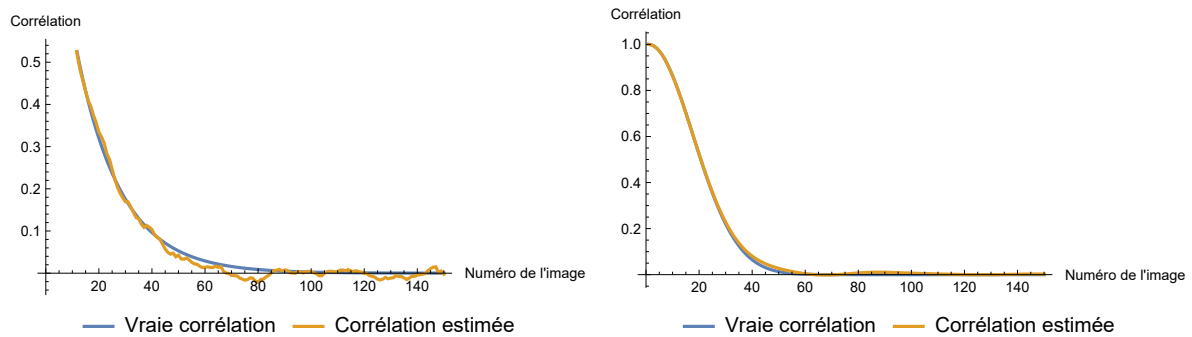


FIGURE 4.2 – Exemple d’une modalité de simulation de tavelures. Ici, la corrélation temporelle est gaussienne.



(A) Corrélation temporelle théorique et estimée d’une séquence temporelle de 150 simulations de tavelures pour une corrélation exponentielle.

(B) Corrélation temporelle théorique et estimée d’une séquence temporelle de 150 simulations de tavelures pour une corrélation gaussienne.

Chapitre 5

Évaluation du modèle

Pour évaluer le modèle, quelques points sont à étudier. Premièrement, on va évaluer les performances pour des images dites *idéales*, donc une tache laser infinie. Comme le réseau donne le temps caractéristique de corrélation, une métrique de log erreur absolue sera utilisé, soit :

$$LEA = |\log \hat{\tau}_c - \log \tau_c|.$$

Pour une même combinaison de paramètres de simulations, mais pour différentes réalisations, on va aussi effectuer un test t de Student (ou une variante selon la distribution des écarts) pour vérifier si le biais du modèle est statistiquement nul, à un seuil de 5%. Une troisième métrique est le score R^2 entre les estimations et les vraies valeurs pour différents temps caractéristiques. Essentiellement, étape par étape, on a :

1. Entraînement du modèle ;
2. Évaluation du modèle :
 - a) Itérer sur un ensemble de validation avec des paramètres identiques et différents,
 - b) Calculer la LEA pour tous les éléments,
 - c) Calculer le score R^2 sur LEA pour tous les éléments ;
 - d) Faire des tests t pour tous les éléments ayant les mêmes propriétés.

De cette manière, on obtient de l'information sur la justesse générale du modèle avec le LER , mais aussi de l'information sur la variance expliquée avec le score R^2 et sur le biais statistique du modèle avec les tests t .

Chapitre 6

Tâches à accomplir

6.1 Génération des données

La première tâche à accomplir consiste à générer plusieurs jeux de données ayant des paramètres de simulation différents, mais aussi identiques. Étant donné la nature statistique et probabiliste des simulations, différentes réalisations ayant les mêmes paramètres donnent des séquences différentes. Ainsi, en donnant plusieurs séquences de paramètres identiques, le réseau n'apprend pas que pour une seule séquence, mais apprend une généralisation.

Le code pour générer les données selon l'Algorithme 3 existe déjà, bien que sous une forme qui pourrait être plus élégante, comme utiliser le paradigme orienté objet. Quelques heures seraient mises pour finaliser et tester un code complet. Le code devrait permettre d'itérer sur plusieurs paramètres, faire quelques simulations par combinaison et enregistrer les données sur disque. L'approche de génération par GPU sera préconisée, car elle accélère grandement l'exécution du code.

6.2 Développement du pipeline d'entraînement : DataLoader et Dataset

Pour bien tirer profit de *PyTorch*, il sera important d'implémenter des classes permettant de charger les données de manière aléatoire parmi les combinaisons de paramètres et de former des lots pour l'entraînement. La documentation web explique bien comment procéder. *PyTorch* gère la sdispersion en mini-lots avec la possibilité d'utiliser des travailleurs parallèles pour accélérer le processus. Pour le Dataset interne du DataLoader, il est implémenté maison, basé sur la version vanille de *PyTorch*.

6.3 Développement du pipeline d'entraînement : Modèle

Le premier modèle présenté plus tôt dans ce document sera implémenté à l'aide de *PyTorch*. L'implémentation initiale ne devrait pas être trop longue. Ce qui risque de demander plus de temps est l'optimisation des hyperparamètres. Toutefois, lorsque l'architecture de base sera présente, modifier les hyperparamètre revient majoritairement à remplacer quelques bouts de code. La plateforme ICE de Valéria est utilisée pour effectuer le travail, tirant profit de son puissant hardware.

6.4 Décision du modèle

Après optimisation des hyperparamètres, il sera décidé de quelle configuration choisir. Si les résultats ne sont pas intéressants et que plusieurs combinaisons d'hyperparamètres ont été explorées, le deuxième modèle sera considéré. Le principal problème avec le second modèle est qu'il n'est pas faisable facilement avec *PyTorch*, on doit construire la mécanique interne. La présence d'un répertoire GitHub est intéressante, mais la qualité de l'implémentation peut être problématique.

6.5 Exploration complémentaire (si le temps le permet)

Dans le cas où le projet avance très bien et qu'il reste suffisamment de temps, le modèle pourrait être modifié pour changer la sortie du réseau. Pour l'instant, une sortie scalaire est considérée, mais une carte locale de temps de corrélation serait plus précise pour des processus dynamiques qui ne sont pas stationnaire dans l'espace, comme différents tissus biologiques ayant différentes rigidités.

Chapitre 7

Expérimentations initiales

L'expérimentation initiale s'est principalement faite en deux étapes. Dans un premier temps, plusieurs séquences de paramètres statistiques et physiques différents sont fournies au modèle. Or, ce ne s'est pas avéré efficace pour tester la capacité d'apprentissage du modèle : les performances étaient très mauvaises et le réseau ne convergeait pas vers une solution à faible perte. On peut expliquer les problèmes par la haute dimensionnalité de l'espace des paramètres physiques et statistiques. En effet, on fait varier le temps caractéristique τ_c sur une plage dynamique importante, de même que pour le temps d'intégration T . On a alors un sous ensemble non négligeable de $\mathbb{R}^2$ où le réseau doit apprendre l'interdépendance des images et des paramètres. De plus, on fait aussi varier la fonction de corrélation temporelle entre une exponentielle et une gaussienne. Le problème devient rapidement lourd, surtout en mémoire. On ne peut pas efficacement apprendre seulement sur de petits sous-ensembles de l'espace des paramètres.

7.1 Sur-apprentissage

Pour la deuxième étape, il a été nécessaire de retourner à la base. Il est important en apprentissage automatique de déterminer si réellement un réseau peut apprendre. On doit donc s'assurer que le réseau puisse sur-apprendre un petit jeu de données. Pour y arriver, le modèle initial est modifié pour considérer le temps d'intégration comme une caractéristique fournie au réseau récurrent. De plus, une restriction sur le domaine des paramètres de simulation est requise. On fait seulement varier le temps caractéristique pour un temps d'intégration spécifique, ici fixé à 1 (les unités sont arbitraires). Le temps caractéristique est divisé en trois modalités : plus petit que T , même ordre que T et plus grand que T . On divise aussi l'entraînement selon la fonction de corrélation. Ces deux séparations permettent de déterminer si une modalité de temps caractéristique ou une fonction de corrélation cause un entraînement

problématique. Voici la démarche de sur-apprentissage d'un petit jeu de données :

— Pour $\tau_c < T$:

1. Générer 8 τ_c dans un espace linéaire de 10^{-2} à 0.5 ;
2. Simuler avec répétition de 2 pour chaque τ_c avec corrélation exponentielle ;
3. Entraîner sur 300 époques ;
4. Simuler avec répétition de 2 pour chaque τ_c avec corrélation gaussienne ;
5. Entraîner sur 300 époques ;

— Pour $\tau_c \approx T$:

1. Générer 8 τ_c dans un espace linéaire de 0.8 à 1.2 ;
2. Simuler avec répétition de 2 pour chaque τ_c avec corrélation exponentielle ;
3. Entraîner sur 300 époques ;
4. Simuler avec répétition de 2 pour chaque τ_c avec corrélation gaussienne ;
5. Entraîner sur 300 époques ;

— Pour $\tau_c > T$:

1. Générer 8 τ_c dans un espace linéaire de 1.5 à 20 ;
2. Simuler avec répétition de 2 pour chaque τ_c avec corrélation exponentielle ;
3. Entraîner sur 300 époques ;
4. Simuler avec répétition de 2 pour chaque τ_c avec corrélation gaussienne ;
5. Entraîner sur 300 époques ;

Il est aussi important d'avoir une optimisation adéquate lors de l'entraînement. Le test de sur-apprentissage s'est effectué à l'aide de l'optimiseur Adam[10] avec deux taux d'apprentissage. On a utilisé un taux de 10^{-3} , puis un taux de 10^{-4} . La perte est la perte $L1$, mais appliquée sur le logarithme de la sortie et de la cible, donc la log erreur absolue (LEA). Comme les temps caractéristiques sont généralement petits, utiliser le logarithme permet d'avoir une échelle intéressante limitant la dissipation du gradient. La figure 7.1 montre les pertes en entraînement pour le régime $\tau_c < T$, et ce pour les deux fonctions de corrélations et les deux taux d'apprentissage. Dix entraînements pour un même ensemble de paramètres physiques ont été faits, permettant de confirmer l'entraînement et non simplement de la chance pour un essai unique. On remarque que les deux corrélations donnent un comportement similaire d'entraînement, ce qui laisse croire qu'il n'y a pas de difficulté spécifique à l'une ou l'autre. On remarque aussi qu'un taux d'apprentissage plus élevé, soit 10^{-3} donne un entraînement plus rapide, convergeant vers de plus petites pertes avant le taux de 10^{-4} . La figure 7.2 montre la moyenne des quatre cas étudiés à la figure 7.1, permettant de mieux visualiser la meilleure performance du taux 10^{-3} , mais aussi le comportement similaire des deux fonctions de corrélation, bien que le cas gaussien semble devenir plus difficile à apprendre à partir de l'époque 200.

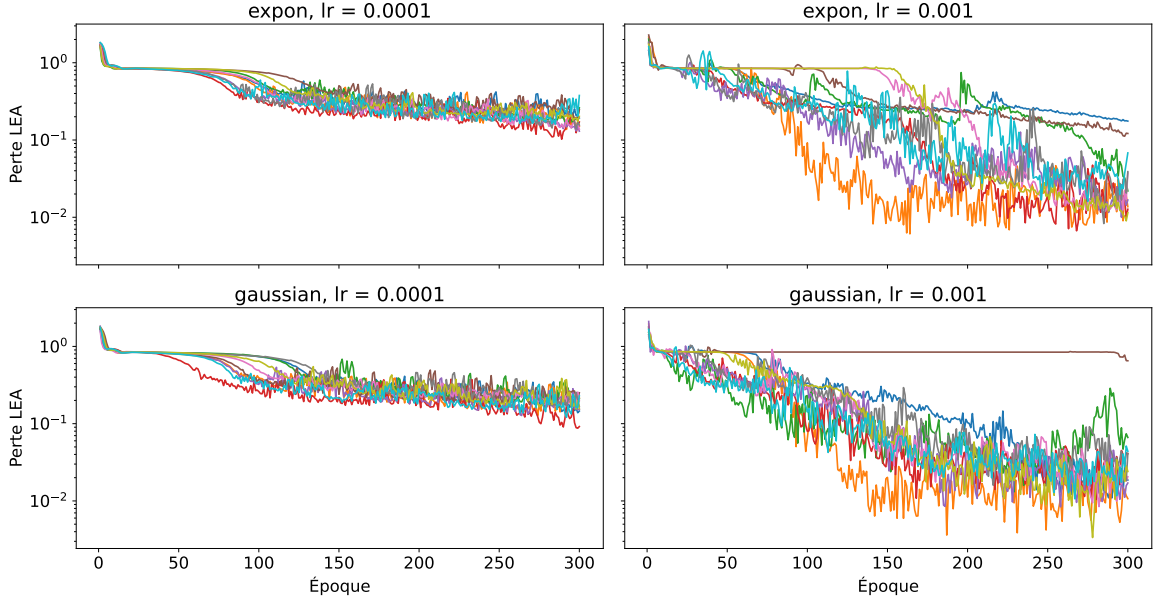


FIGURE 7.1 – Graphiques des courbes d’entraînement pour les principaux paramètres physiques du régime $\tau_c < T$. Sur une même ligne, on compare l’effet du taux d’apprentissage.

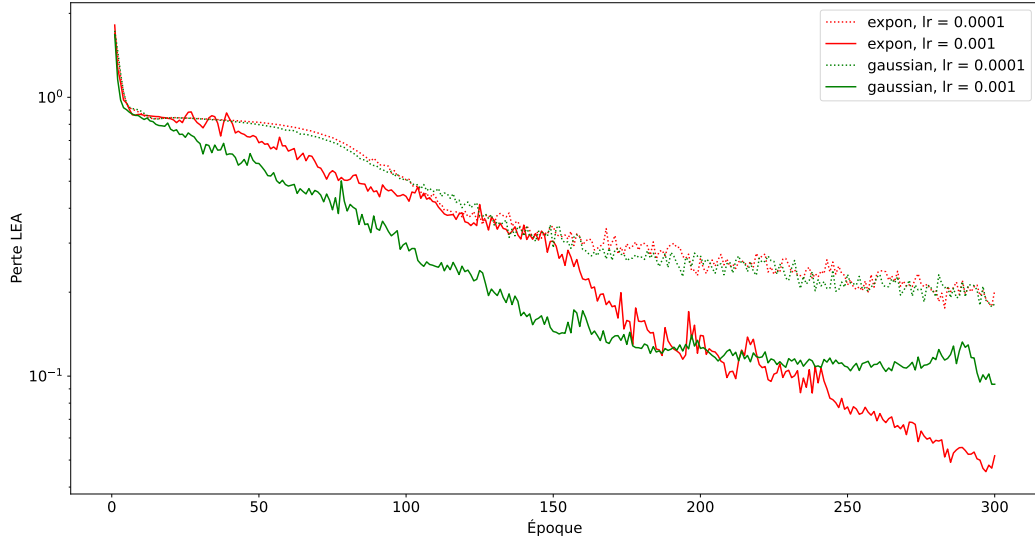


FIGURE 7.2 – Moyennes des dix essais d’entraînement pour les quatre configurations importantes pour le régime $\tau_c < T$.

Pour le régime $\tau_c \approx T$, on a aussi effectué dix entraînements avec les mêmes taux d'apprentissage, autant pour une corrélation exponentielle que gaussienne. Les dix courbes d'entraînement sont présentées à la figure 7.3, alors que les moyennes sont présentées à la figure 7.4. On remarque une convergence rapide dans les quatre cas, mais aussi qu'un taux d'apprentissage plus petit serait mieux.

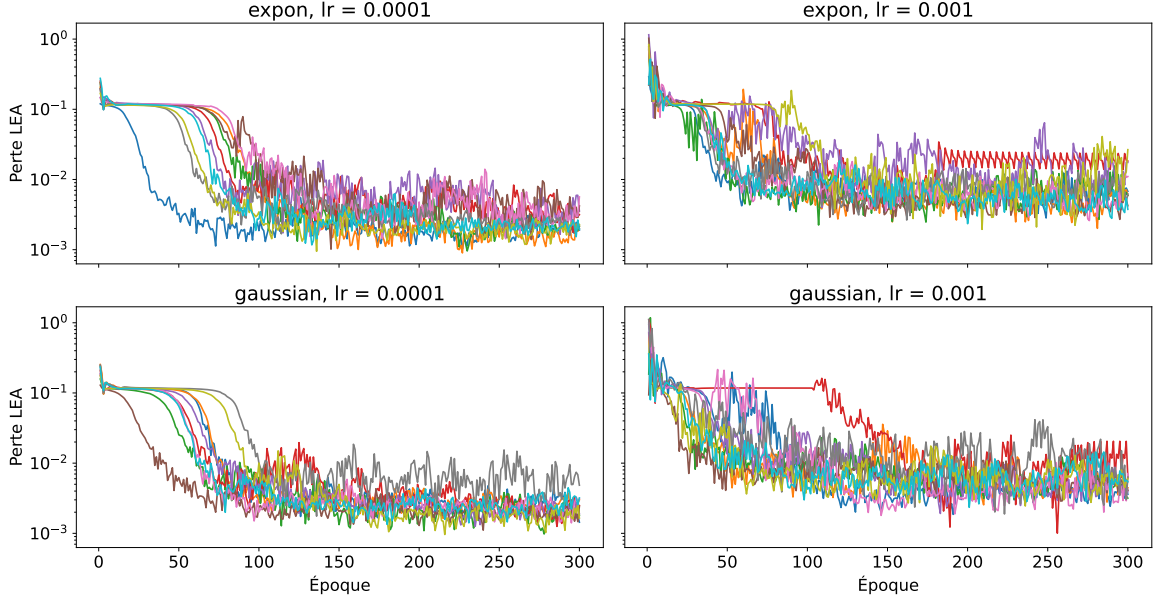


FIGURE 7.3 – Graphiques des courbes d'entraînement pour les principaux paramètres physiques du régime $\tau_c \approx T$. Sur une même ligne, on compare l'effet du taux d'apprentissage.

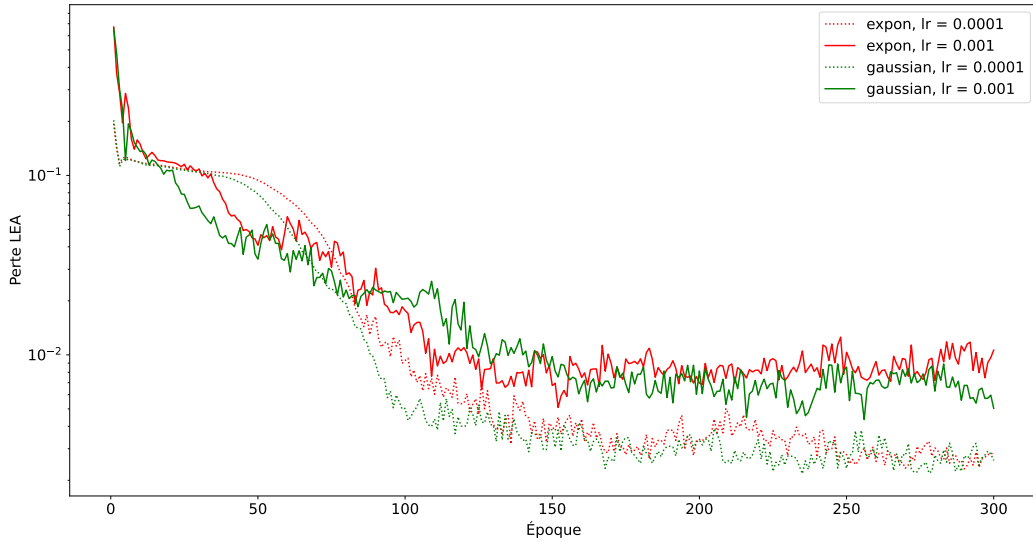


FIGURE 7.4 – Moyennes des dix essais d'entraînement pour les quatre configurations importantes pour le régime $\tau_c \approx T$.

Finalement, les courbes des dix entraînements du régime $\tau_c > T$ sont présentées à la figure 7.5 et la figure 7.6 présente les moyennes. Les courbes sont similaires à celles du régime précédent, notamment avec un taux de 10^{-4} plus efficace sur le long terme.

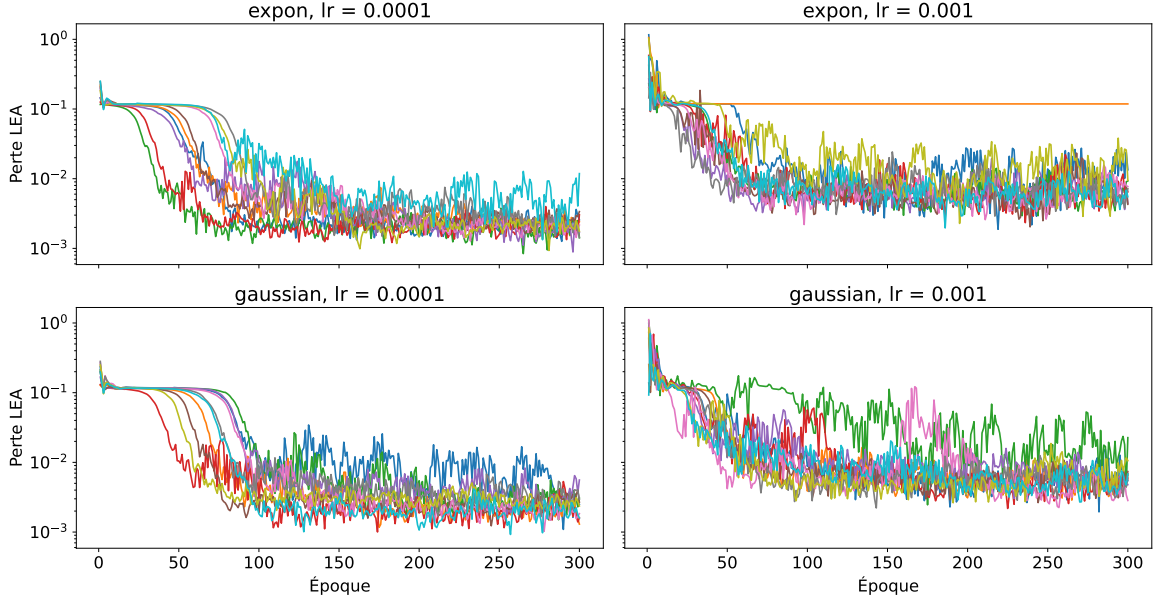


FIGURE 7.5 – Graphiques des courbes d’entraînement pour les principaux paramètres physiques du régime $\tau_c > T$. Sur une même ligne, on compare l’effet du taux d’apprentissage.

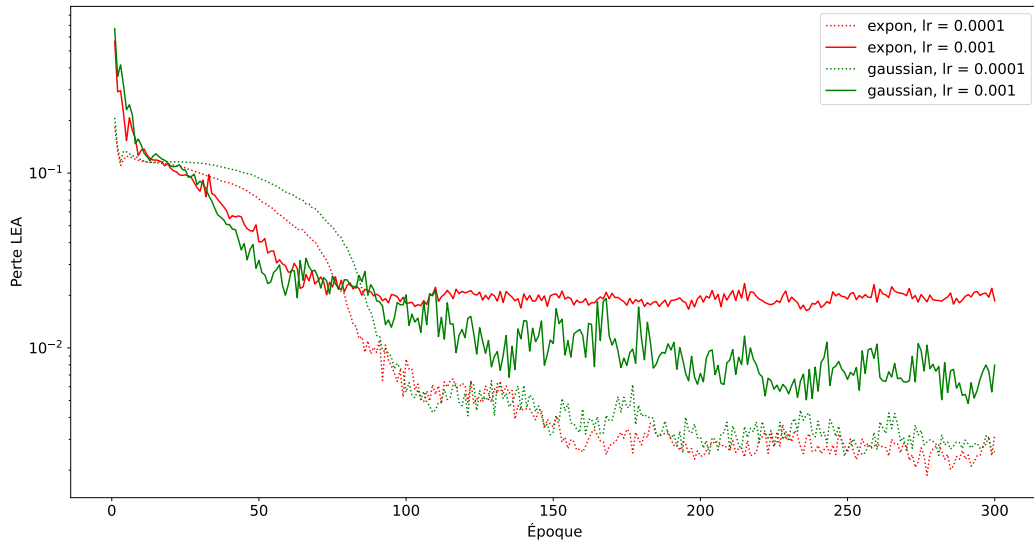


FIGURE 7.6 – Moyennes des dix essais d’entraînement pour les quatre configurations importantes pour le régime $\tau_c > T$.

Finalement, il est aussi pertinent de vérifier si un taux d'apprentissage de 10^{-2} serait viable. On a alors expérimenté avec le régime $\tau_c < T$, car c'est lui qui pourrait bénéficier d'un taux d'apprentissage plus élevé. On a alors entraîné sur 200 époques pour le même nombre de données et pour dix essais indépendants. La figure 7.7 montre les dix courbes d'entraînement, ainsi que la moyenne. Il est évident par la figure que ce taux d'entraînement n'est pas à utiliser, du moins sur toutes les époques. On plafonne rapidement à une perte autour de 0.85, ce qui n'est pas optimal comparativement aux taux plus petits utilisés précédemment. Le code utilisé pour effectuer ces surapprentissages est présent dans le fichier

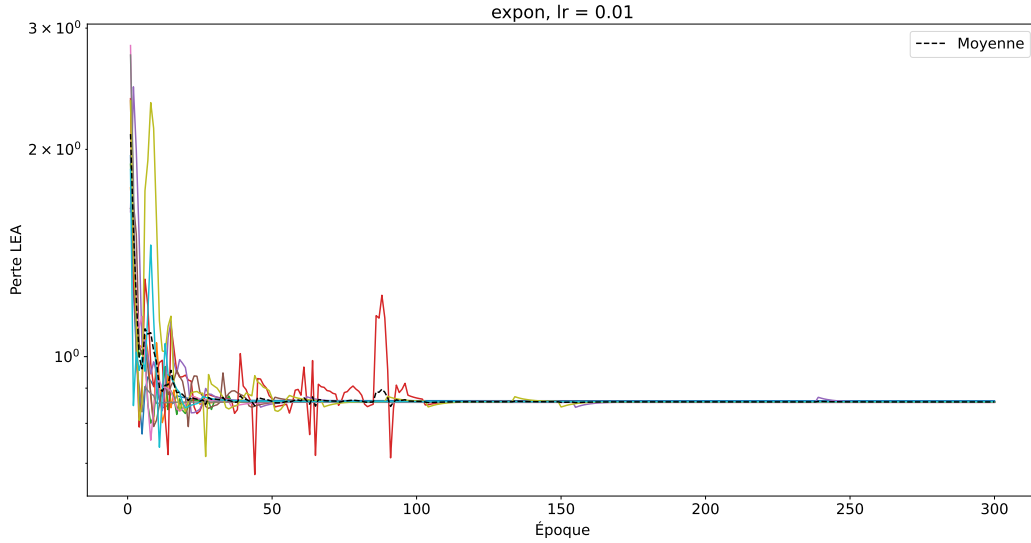


FIGURE 7.7 – Courbes et moyenne des dix essais d'entraînement dans le régime $\tau_c < T$ pour un taux d'apprentissage de 10^{-2} et une corrélation exponentielle.

`projetIntegrateur/overfit_trials.py` du répertoire GitHub, alors que les images sont produites à partir du fichier `projetIntegrateur/overfit_analysis.py`.

En terminant ce chapitre, il est pertinent de parler de l'interprétation de la perte LEA , soit la log erreur absolue. Pour rappel, si τ_c est le vrai temps caractéristique et $\hat{\tau}_c$ est l'estimation du réseau, la perte LEA est définie comme

$$LEA = |\log \hat{\tau}_c - \log \tau_c| = \left| \log \left(\frac{\hat{\tau}_c}{\tau_c} \right) \right|$$

et on peut revenir à l'échelle naturelle avec :

$$E = \exp\{\pm LEA\}$$

où E est l'erreur multiplicative entre $\hat{\tau}_c$ et τ_c . Par exemple, si on a $LEA = 1$, on a $E = \exp\{\pm 1\}$, donc l'erreur multiplicative est soit d'environ 2.72 ou 0.37, donc que $\frac{\hat{\tau}_c}{\tau_c} \approx 2.72$ ou $\frac{\hat{\tau}_c}{\tau_c} \approx 0.37$. Dans les cas présentés jusqu'ici, on arrive presque tout le temps à des pertes finales sous 10^{-1} (voire 10^{-2}), ce qui donne des erreurs multiplicatives de moins de $\exp\{\pm 0.1\}$, soit

entre 0.9 et 1.1. Si on prend un τ_c de 0.5, on a alors une estimation dans l'intervalle $[0.45, 0.55]$, ce qui est très bien pour un intervalle conservateur compte tenu de la complexité stochastique des tavelures laser. Le réseau est capable d'apprendre une petite séquence. Il faut maintenant tester l'entraînement et la validation sur un jeu de données complet.

Chapitre 8

Résultats initiaux

Ce chapitre présente les principaux résultats d’entraînement obtenus durant la première partie du projet. Dans tous les cas présentés, on utilise le réseau schématisé à la figure 3.2, mais avec des hyperparamètres différents.

8.1 Jeux de données

Pour ce chapitre, deux jeux de données sont utilisés, simulés selon ce qui est présenté au chapitre 4. Premièrement, un lot de 500 séquences temporelles de 50 images 128×128 est généré. Dans ces simulations, on a des tavelures de largeur moyenne de 3 pixels et des temps de corrélations variant en logarithme entre 10^{-2} et 10, pour un temps d’intégration de 1 (toutes ces unités sont arbitraires, tant qu’elles sont les mêmes). Échantillon 100 valeurs de τ_c uniformément dans l’espace logarithmique et génère 5 séquences ayant le même τ_c , d’où la taille finale de 500 séquences. La fonction de corrélation est restée constante à la fonction exponentielle.

Deuxièmement, un lot de 5000 séquences temporelles de 50 images 128×128 est généré. Les tavelures sont aussi de largeur moyenne de 3 pixels, avec des temps de corrélations dans le même espace logarithmique. Cependant, on échantillonne 1000 valeurs différentes de τ_c avec encore une fois 5 répétitions par τ_c , donnant 5000 séquences. Le temps d’intégration et la fonction de corrélation n’ont pas changés.

Dans les deux cas, on génère les séquences à l’aide du fichier `projetIntegrateur/data_generation.py`, qui utilise notamment le fichier `projetIntegrateur/src/simulations/time_integrated_sims.py` avec les bons paramètres. Pour les sections qui suivent, le code de l’entraînement des différents réseaux est présent dans le fichier `projetIntegrateur/base_model_testing.py` du répertoire GitHub. Le fichier `projetIntegrateur/base_model_analysis.py` sert à produire les images.

8.2 Premier réseau

Le premier réseau est composé d'un CNN ayant trois couches, chacune donnant respectivement une carte de caractéristique de 16, 32 et 64 canaux. Chaque couche est composée d'un noyau 3×3 , suivi d'une activation ReLU. On applique un rembourrage des images avec un mode de réflexion, permettant de garder la taille initiale des images. Le mode de réflexion est choisi pour ne pas tenir compte de données fausses, comme le mode constant, et est logique avec la nature stochastique des tavelures, permettant de répéter les motifs par réflexion. À la fin du CNN, on applique une convolution 1×1 permettant de ramener le nombre de canaux à 1, et on passe le tout dans une couche linéaire diminuant le nombre de caractéristiques à 64 par image, permettant une sorte d'attention spatiale pouvant apprendre.

Pour la partie récurrente, en plus des 64 caractéristiques par images, on en ajoute une : le temps d'intégration. On donne ces caractéristiques à un GRU unidirectionnel d'une couche et 128 unités cachées. Le tout donne finalement un scalaire, soit le logarithme naturel du temps caractéristique de corrélation estimé. À la figure 8.1, on montre les courbes d'apprentissage pour ce réseau. On a utilisé le jeu de 500 séquences temporelles de 50 images, séparées en données d'entraînement (80%) et de validation (20%). L'apprentissage se fait par mini lots de taille 8 et à l'aide d'un optimiseur Adam auquel on spécifie un taux d'apprentissage de 10^{-4} qui reste constant. On note un fort sur-apprentissage, car la perte en validation atteint rapidement un plateau. Le second réseau sera modifié pour tenter de cibler la cause de la mauvaise généralisation.

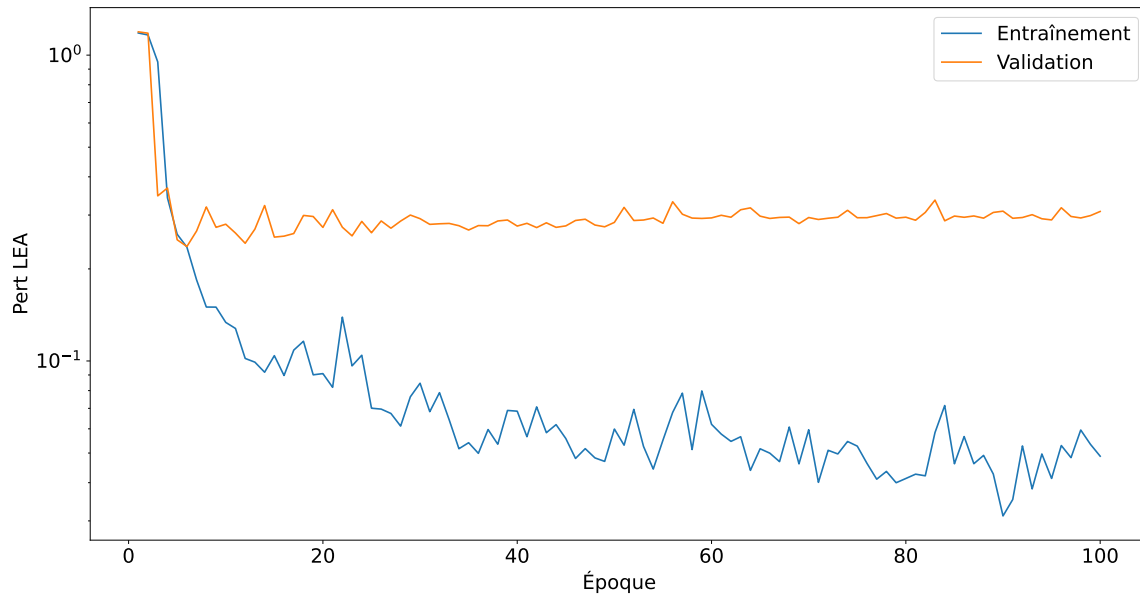


FIGURE 8.1 – Courbes d'entraînement et de validation pour le premier réseau. On entraîne sur 100 époques.

8.3 Deuxième réseau

Le second réseau diffère du premier dans le nombre de canaux et de couches du CNN, du nombre de neurones dans la partie linéaire d'attention et dans le nombre d'unités cachées du GRU. On a deux couches CNN, toutes deux avec 8 canaux de sortie. Pour la couche linéaire, on a 32 sorties au lieu de 64. Le GRU contient 64 unités cachées. Le but de ce réseau est de vérifier si le premier réseau était trop complexe pour les données, donc favorisait le sur-apprentissage. On garde le même nombre de mini lots et le même optimiseur Adam avec un taux d'apprentissage de 10^{-4} , et on entraîne sur les mêmes séquences. La figure 8.2 montre les courbes d'entraînement pour ce réseau. On remarque des performances similaires, quoique légèrement meilleures avec un plateau plus tard et plus bas, ainsi qu'avec un sur-apprentissage plus lent à démarrer. Cela laisse croire que le modèle peut être rétrécie sans rendre l'apprentissage plus difficile.

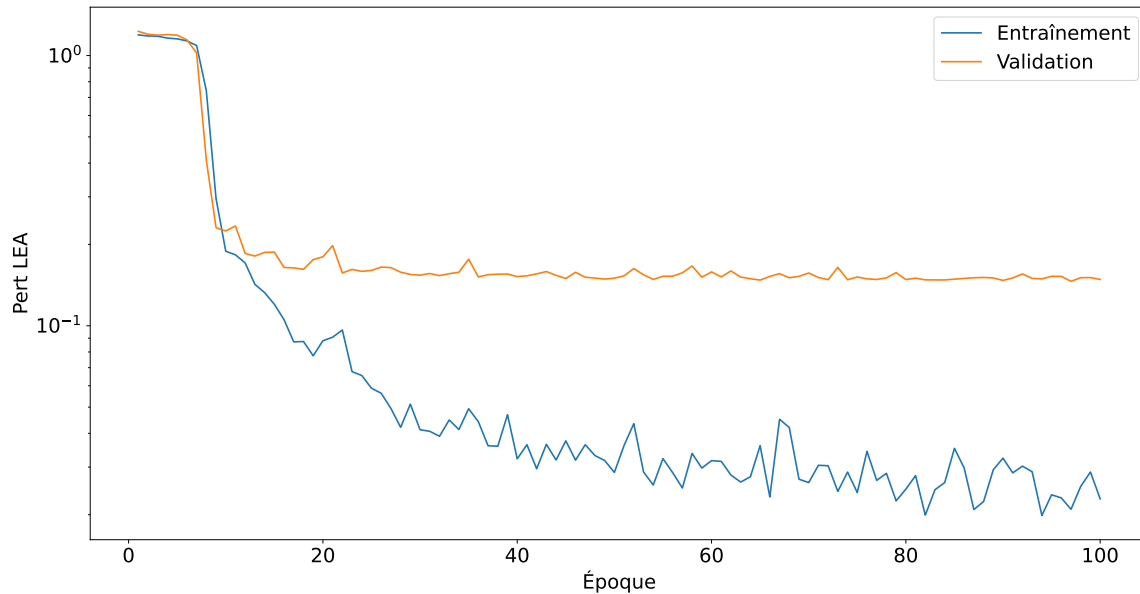


FIGURE 8.2 – Courbes d'entraînement et de validation pour le deuxième réseau. On entraîne sur 100 époques.

8.4 Troisième réseau

Le troisième réseau est aussi une légère réduction du premier. On a une fois de plus deux couches, comme le second, mais cette fois les canaux sont 8, puis 16. On ne change pas l'attention ni le GRU. En revanche, on utilise le jeu de 5000 données. Une possible explication des mauvaises performances des deux premiers est le manque de données. Étant donné la nature stochastique des tavelures, un plus grand nombre de données pourrait apporter une bonne amélioration. On utilise encore Adam et des mini lots de taille 8, ainsi qu'un taux

d'apprentissage de 10^{-4} . La figure 8.3 présente les courbes d'apprentissage. On remarque un comportement similaire aux deux réseaux précédents, toutefois une performance en validation moins bonne que ce qu'on pourrait espérer.

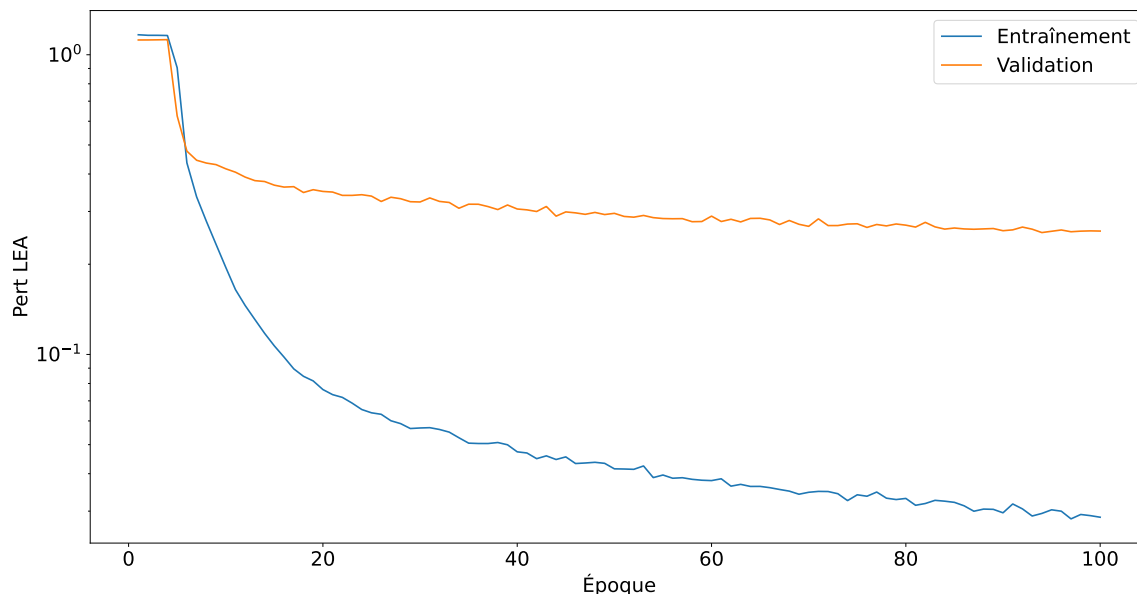


FIGURE 8.3 – Courbes d'entraînement et de validation pour le troisième réseau. On entraîne sur 100 époques.

8.5 Quatrième réseau

Le quatrième réseau est le même que le troisième, mais où on a augmenté le taux d'apprentissage à 10^{-3} . On utilise les mêmes données que le précédent réseau, avec Adam et des mini lots de taille 8. Augmenter le taux d'apprentissage pourrait accélérer l'entraînement et empêcher le réseau de tomber des extremums qui ne sont pas optimaux, donc potentiellement améliorer la validation. La figure 8.4 montre les courbes d'apprentissage du réseau quatre. On note une amélioration non négligeable de la performance en validation avec une perte variant autour de 0.1, ce qui laisse croire qu'augmenter le taux d'apprentissage permet de meilleurs résultats. On note toutefois d'assez importantes variations de la perte d'une époque à l'autre. Ces variations peuvent être causées par un taux d'apprentissage un peu trop élevé. Avoir un taux entre 10^{-4} et 10^{-3} pourrait s'avérer intéressant.

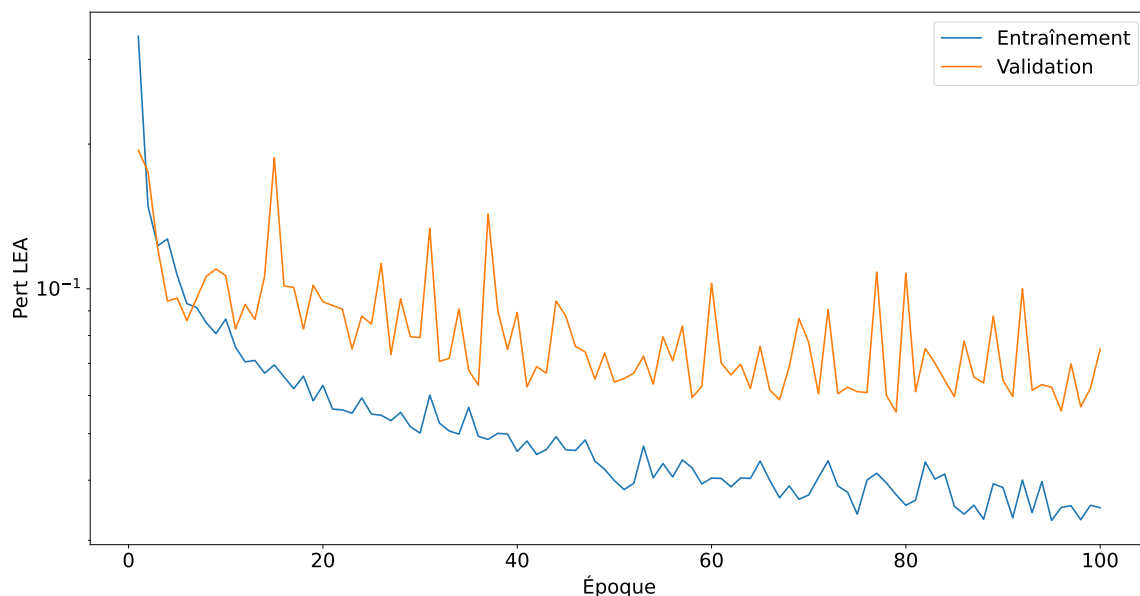


FIGURE 8.4 – Courbes d’entraînement et de validation pour le quatrième réseau. On entraîne sur 100 époques.

8.6 Cinquième réseau

Le cinquième et dernier réseau présenté dans ce chapitre est le même que le précédent, mais où on réduit de moitié le taux d’apprentissage, donnant 5×10^{-4} , soit un entre deux pour le troisième et le quatrième réseau. La figure 8.5 montre ses courbes d’apprentissage. On remarque une performance en validation meilleure que les quatre autres réseaux présentés, avec une perte généralement sous la barre de 0.1. On remarque encore quelques fortes variations, donc il resterait de l’optimisation à faire pour adoucir la courbe de validation.

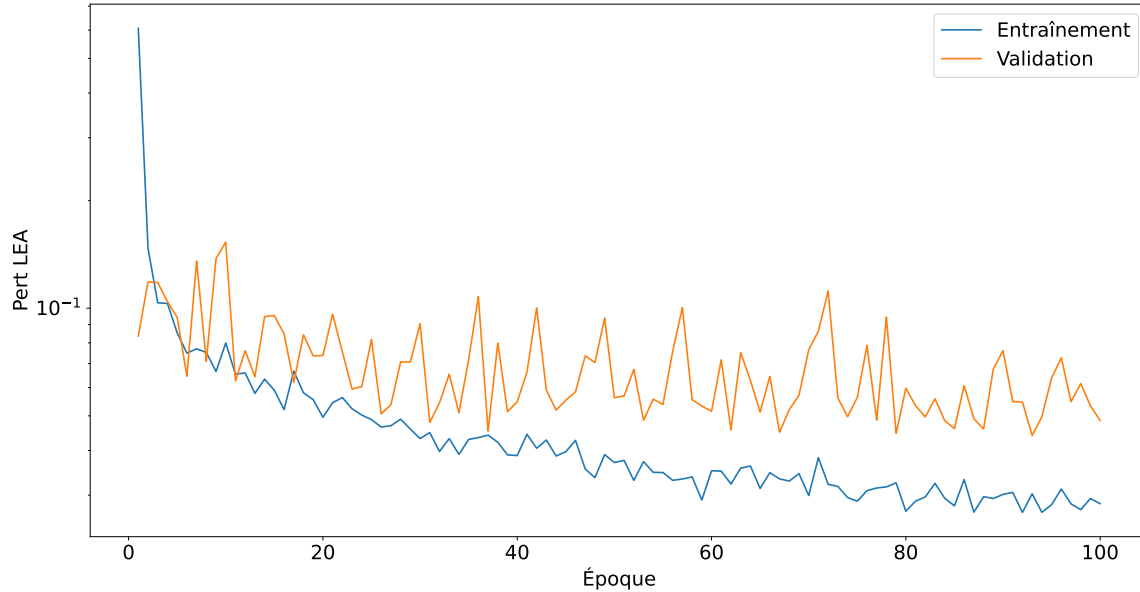


FIGURE 8.5 – Courbes d’entraînement et de validation pour le cinquième réseau. On entraîne sur 100 époques.

8.7 Autres métriques d’analyse

La perte LEA est intéressante pour avoir une vue globale. Par contre, elle ne permet pas vraiment d’évaluer la justesse du réseau dans sa pleine complexité. On a vu au chapitre précédent que selon le ratio T/τ_c , l’apprentissage se fait plus ou moins difficilement. Il faut donc s’intéresser à la précision en fonction du temps caractéristique ciblé.

8.7.1 Premier réseau

Pour le premier réseau, la figure 8.6 montre la log erreur absolue en fonction du τ_c cible. On remarque que l’entraînement se fait bien, avec une perte généralement entre 0.1 et 0.01 pour la plupart des époques affichées. Ce constat n’est pas nouveau, on savait déjà que le réseau avait un apprentissage en entraînement très bon. Pour la validation, ce n’est pas pareil. On remarque une perte plus élevée et qui semble augmenter avec la grandeur des cibles. Ce constat est confirmé à la figure 8.7 qui montre $\hat{\tau}_c$ en fonction de τ_c . La légende contient, en plus de l’époque associée à la couleur de la courbe, le score R^2 , soit la qualité de la régression linéaire entre les prédictions et les cibles. On remarque, en validation, que plus la valeur de la cible augmente, plus le réseau devient imprécis. On peut aussi se concentrer sur la figure 8.8 qui montre aussi $\hat{\tau}_c$ en fonction de τ_c , mais en échelle logarithmique, ce qui peut être plus représentatif de l’apprentissage, car c’est sur cet échelle qu’on calcule la perte après tout. L’avantage de l’échelle logarithmique est principalement de mieux voir le comportement des cibles de petites valeurs. On y remarque une bonne précision, d’où les coefficients de détermination R^2 plus élevés, mais elle diminue avec l’amplitude de la cible.

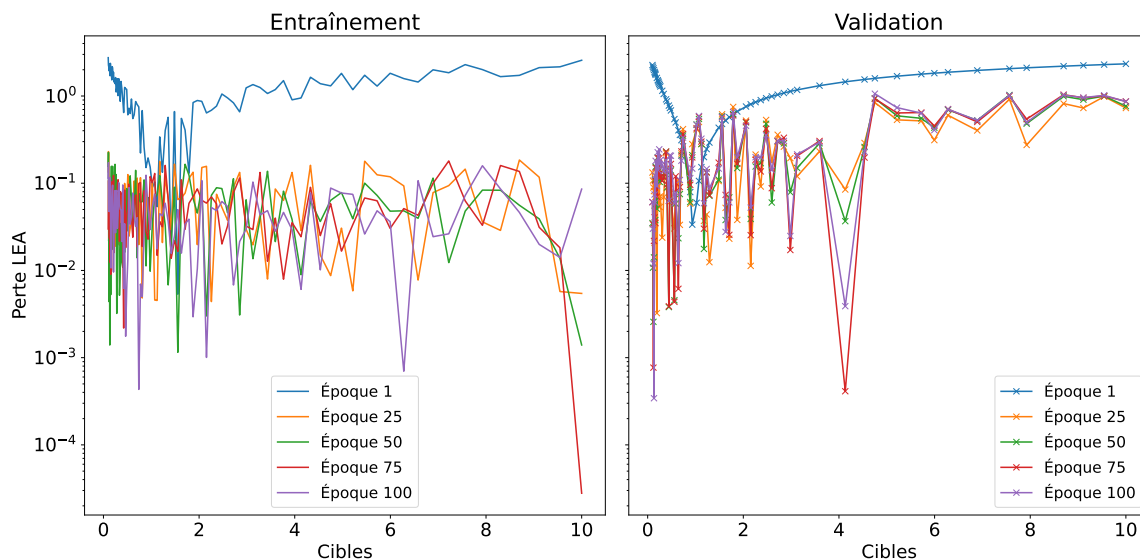


FIGURE 8.6 – Perte log erreur absolue du premier réseau en fonction des cibles pour l’entraînement et la validation. On affiche quelques époques par graphique.

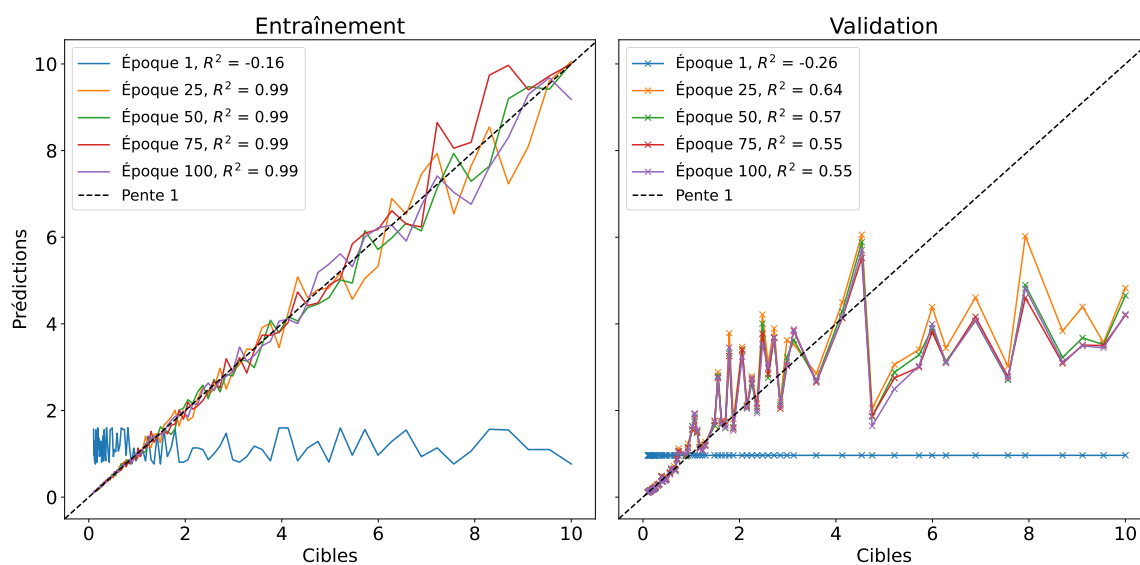


FIGURE 8.7 – Prédictions en fonction des cibles du le premier réseau pour l’entraînement et la validation. On affiche quelques époques par graphique.

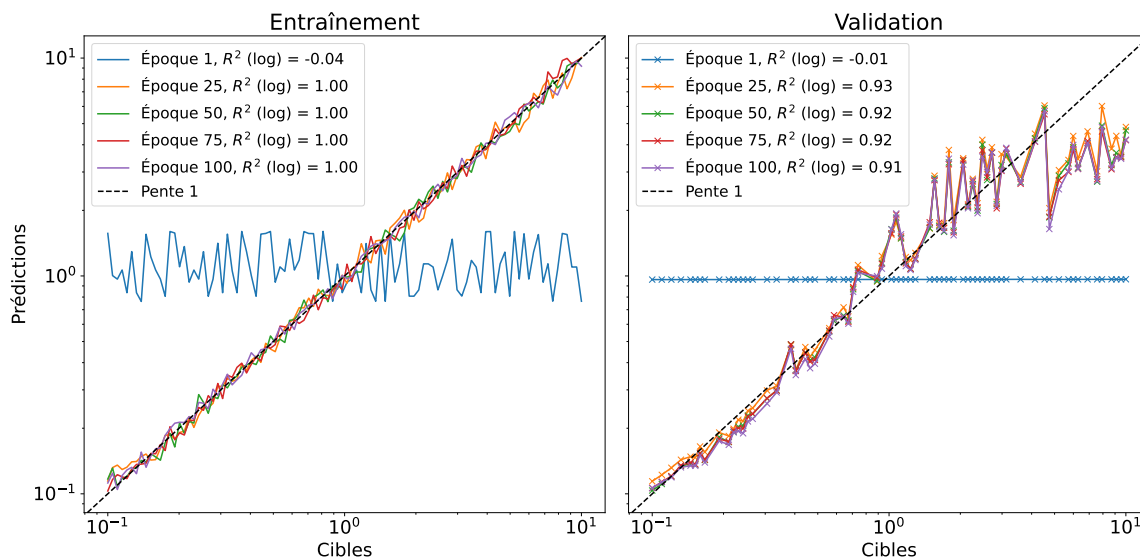


FIGURE 8.8 – Log-prédictions en fonction des log-cibles du le premier réseau pour l’entraînement et la validation. On affiche quelques époques par graphique.

8.7.2 Deuxième réseau

Pour le second réseau, les mêmes métriques sont évaluées. La figure 8.9 montre la perte en fonction de la cible. La principale conclusion des courbes d’entraînement est vérifiée ici, soit qu’un réseau plus petit n’apporte pas, dans ce cas-ci, de dégradation de performances. On y

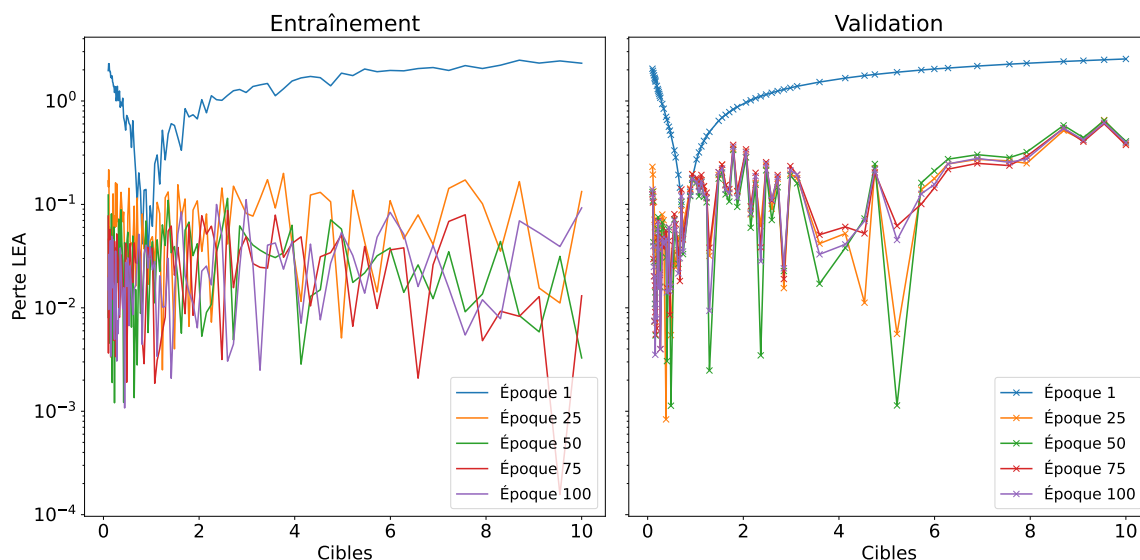


FIGURE 8.9 – Perte log erreur absolue du deuxième réseau en fonction des cibles pour l’entraînement et la validation. On affiche quelques époques par graphique.

voit aussi une certaine perte de performance avec la montée en grandeur de la cible, comme

pour le premier réseau. La figure 8.10, et la figure 8.11 pour l'échelle logarithmique, donnent la même conclusion sur la performance. On y note aussi, dans les deux cas, des coefficients de

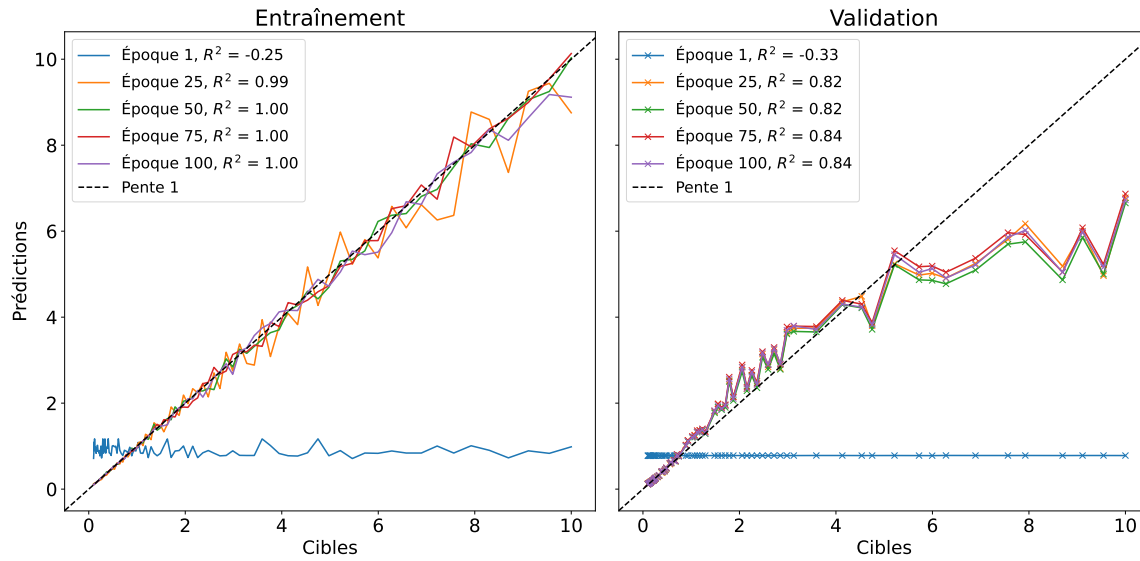


FIGURE 8.10 – Prédications en fonction des cibles du le deuxième réseau pour l'entraînement et la validation. On affiche quelques époques par graphique.

détermination plus élevés que pour le premier réseau. Il est intéressant de constater que ces conclusions ne sont pas évidentes avec les courbes d'entraînement seules, notamment car elles moyennent sur les divers régimes de τ_c . On voit finalement que la linéarité est mieux préservée dans l'espace log pour les hauts temps caractéristique dans ce second cas comparativement au premier cas.

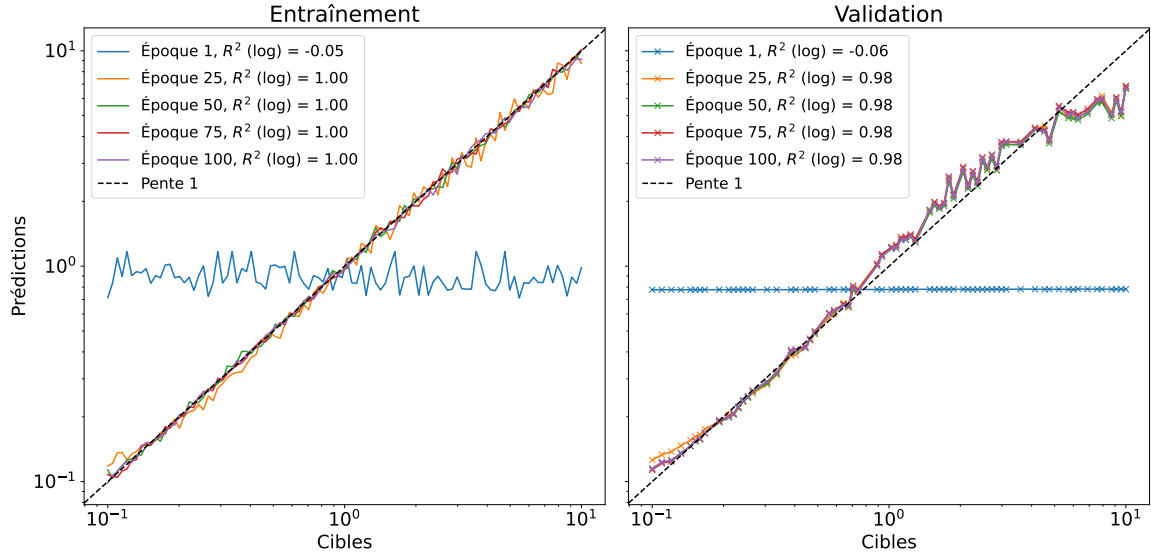


FIGURE 8.11 – Log-prédictions en fonction des log-cibles du le deuxième réseau pour l’entraînement et la validation. On affiche quelques époques par graphique.

8.7.3 Troisième réseau

Pour le troisième réseau, la perte en fonction des cibles est présentée à la figure 8.12. On y voit qu’en tant que tel, ajouter des données n’est pas nécessairement mieux, car on a des performances proches du précédent réseau. Les figure de $\hat{\tau}_c$ en fonction de τ_c , soit 8.13 et 8.14 pour l’échelle log, montrent la même conclusion, notamment par les coefficients de détermination. On peut expliquer une partie des moins bonnes performances par l’augmentation non

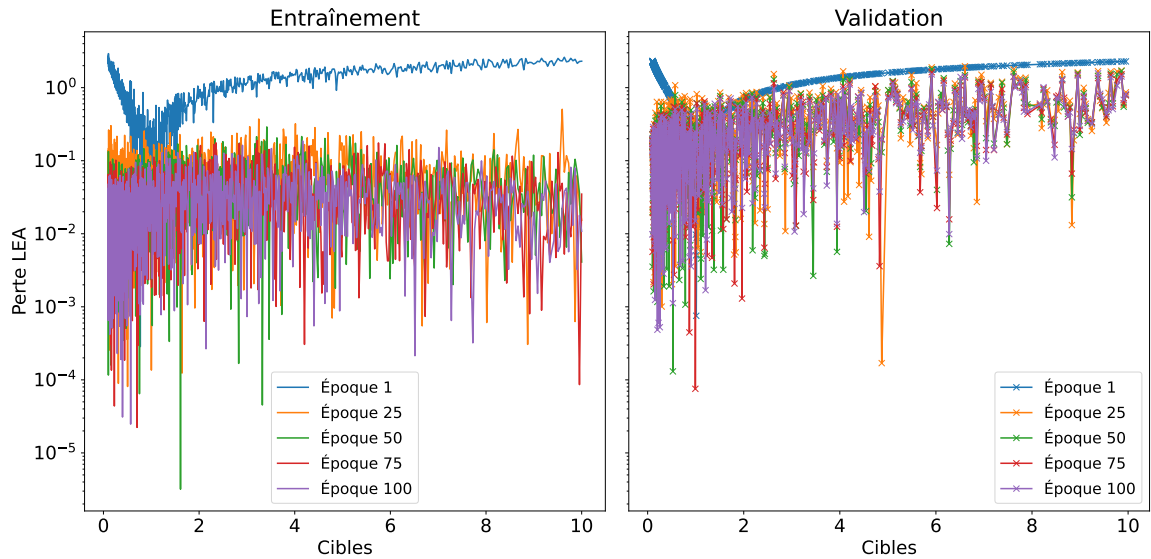


FIGURE 8.12 – Perte log erreur absolue du troisième réseau en fonction des cibles pour l’entraînement et la validation. On affiche quelques époques par graphique.

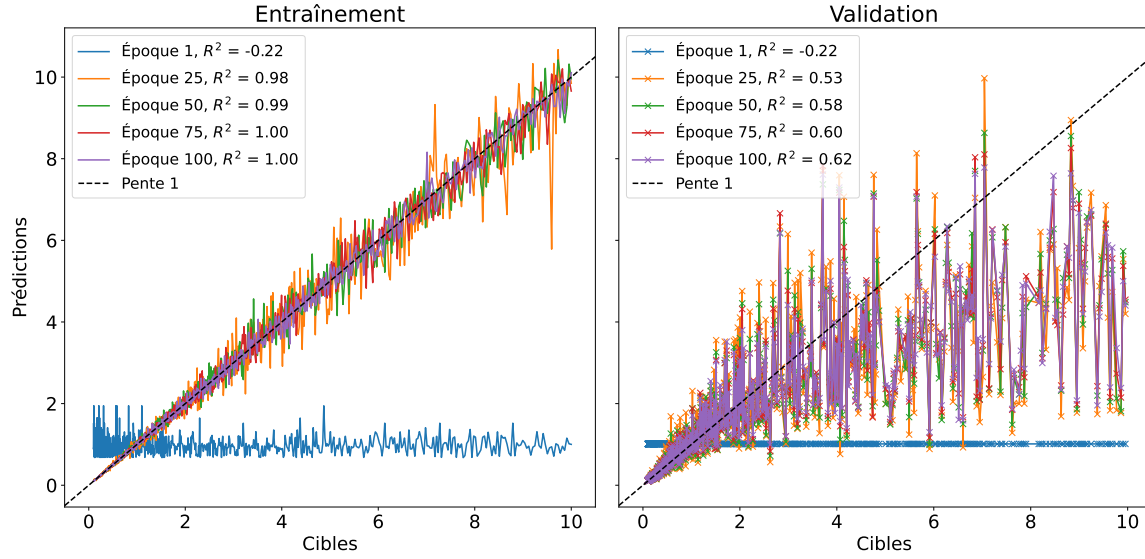


FIGURE 8.13 – Prédictions en fonction des cibles du le troisième réseau pour l’entraînement et la validation. On affiche quelques époques par graphique.

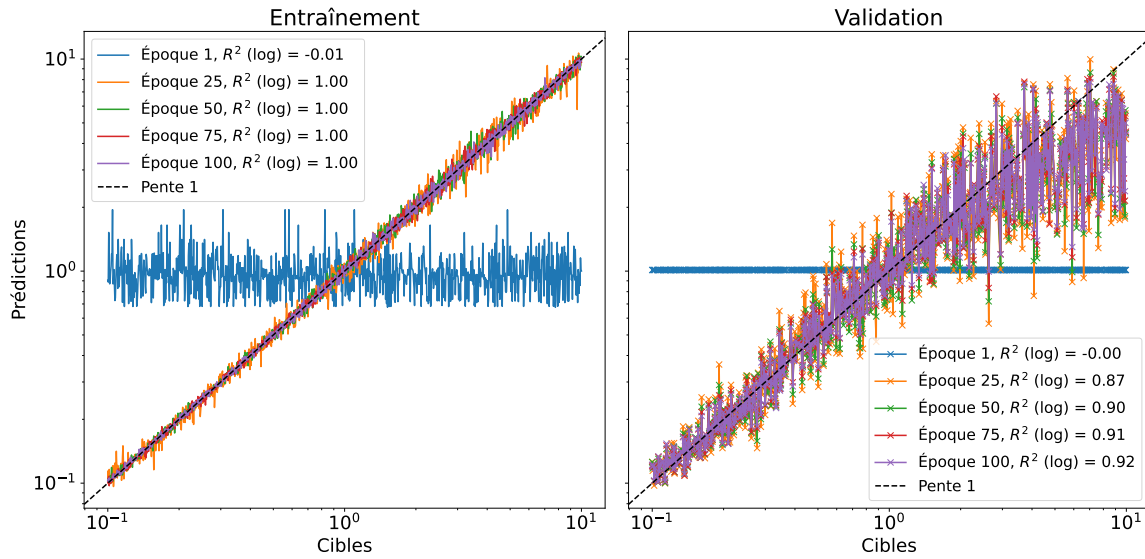


FIGURE 8.14 – Log-prédictions en fonction des log-cibles du le troisième réseau pour l’entraînement et la validation. On affiche quelques époques par graphique.

négligeables de temps caractéristiques dans le régime $\tau_c > T$ pour ce réseau contrairement au deux précédents. Au lieu d’avoir seulement quelques points, on y en retrouve beaucoup plus. Si les performances ne sont pas précises dans ce régime, alors augmenter le nombre de prédictions diminue la qualité générale sans surprise. Il faut alors changer des hyperparamètres du réseau, comme ce qui est fait pour le quatrième.

8.7.4 Quatrième réseau

Pour le quatrième réseau, on prend le troisième, mais on augmente le taux d'apprentissage de 10^{-4} à 10^{-3} . La perte en fonction des cibles est présentée à la figure 8.15. On y voit une nette amélioration en validation avec des pertes plus souvent sous la barre de 0.1, mais ayant encore la remontée pour τ_c qui augmente. La meilleure précision se voit aussi dans la figure

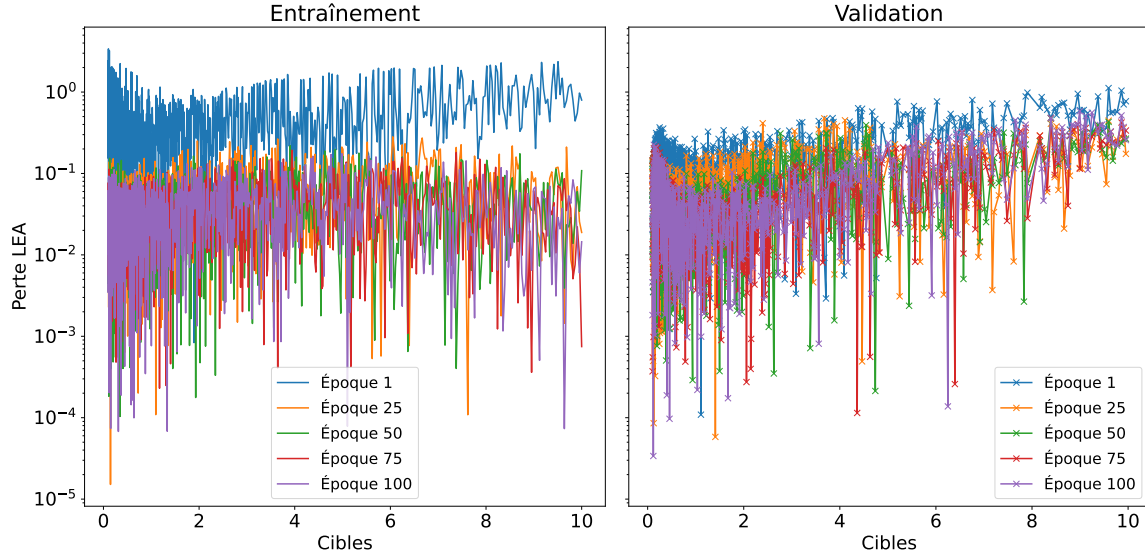


FIGURE 8.15 – Perte log erreur absolue du quatrième réseau en fonction des cibles pour l'entraînement et la validation. On affiche quelques époques par graphique.

8.16 où les coefficients de déterminations arrivent rapidement à dépasser 0.9, la linéarité étant meilleure. Pour l'échelle logarithmique, on s'approche de l'unité, tel que montré à la figure 8.16. Bien qu'il y ait encore une certaine difficulté pour le régime $\tau_c > T$, augmenter le taux d'apprentissage rend cette difficulté plus facile à assimiler.

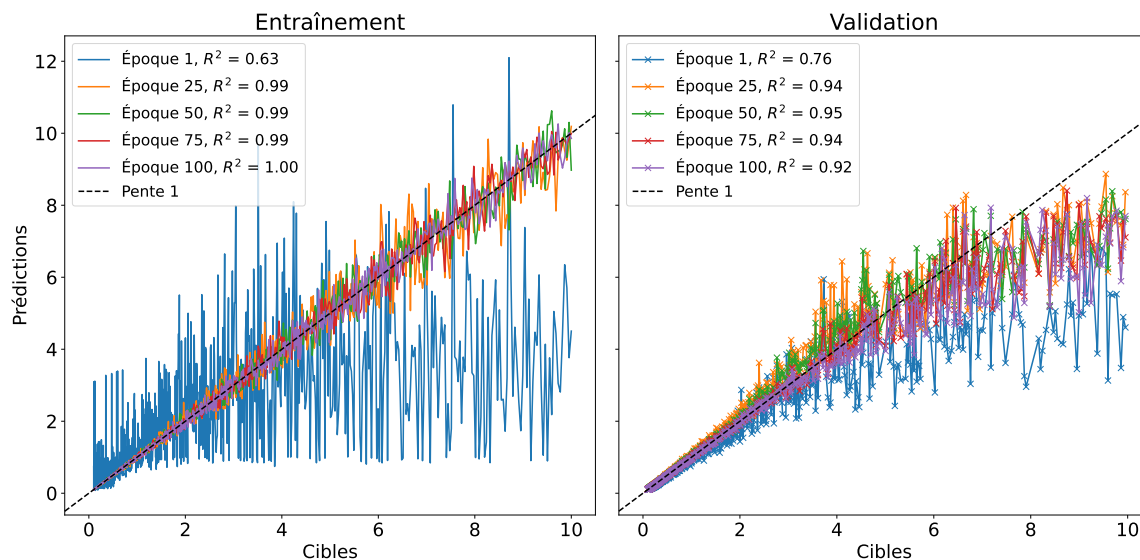


FIGURE 8.16 – Prédictions en fonction des cibles du le quatrième réseau pour l’entraînement et la validation. On affiche quelques époques par graphique.

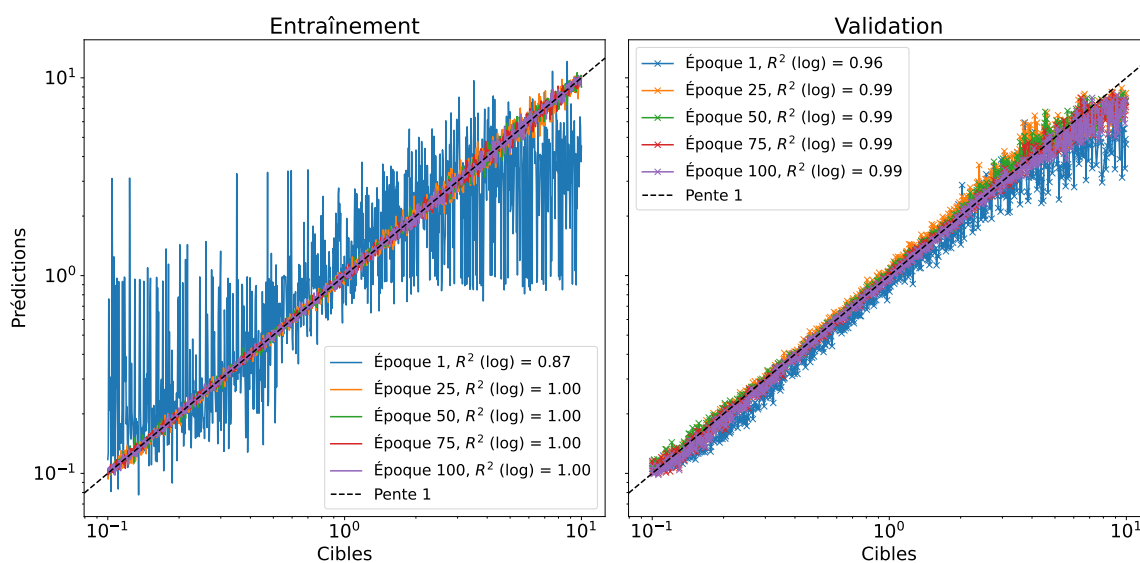


FIGURE 8.17 – Log-prédictions en fonction des log-cibles du le quatrième réseau pour l’entraînement et la validation. On affiche quelques époques par graphique.

8.7.5 Cinquième réseau

Pour le dernier réseau présenté, on se base sur le troisième et le quatrième, mais avec un taux de 5×10^{-4} . Les courbes d'apprentissage semblaient indiquer que le réseau était dans les meilleurs, ce qui est aussi remarqué à la figure 8.18, soit la perte en fonction des cibles. Le régime $\tau_c < T$, ainsi que le régime $\tau_c \approx T$ sont souvent sous la barre de 0.01 de perte, alors que le régime $\tau_c > T$ semble plus stable et varier légèrement autour de 0.1. À la figure

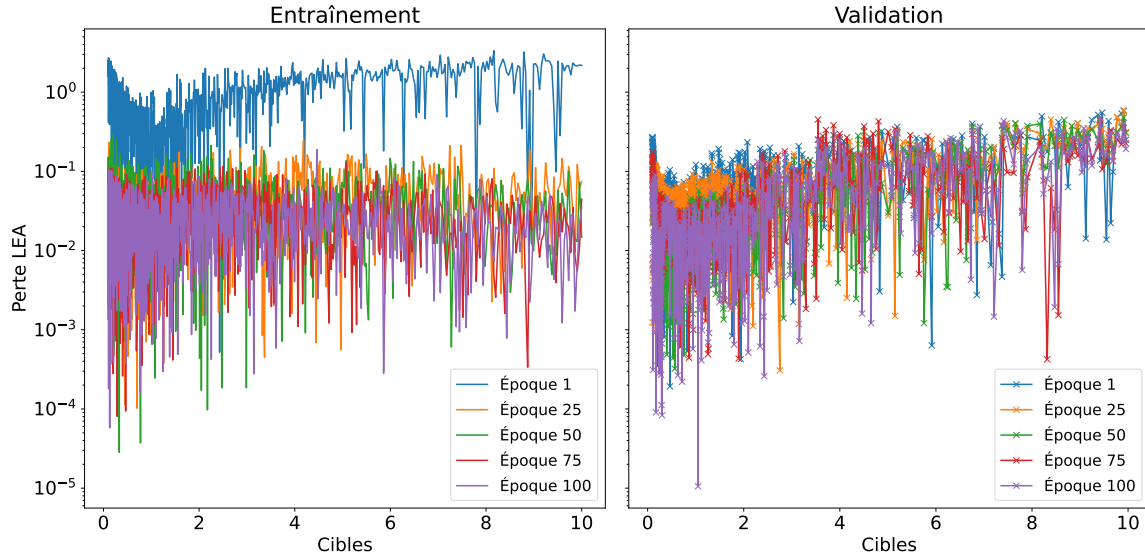


FIGURE 8.18 – Perte log erreur absolue du cinquième réseau en fonction des cibles pour l'entraînement et la validation. On affiche quelques époques par graphique.

8.19, on voit que les prédictions restent proches des cibles sur un intervalle de τ_c plus grand, déviant plus lentement et à plus petite amplitude que pour les réseaux précédents, idem pour la figure en échelle logarithmique, la figure 8.20. On est plus centré sur la diagonale, et ce avec une variance plus faible.

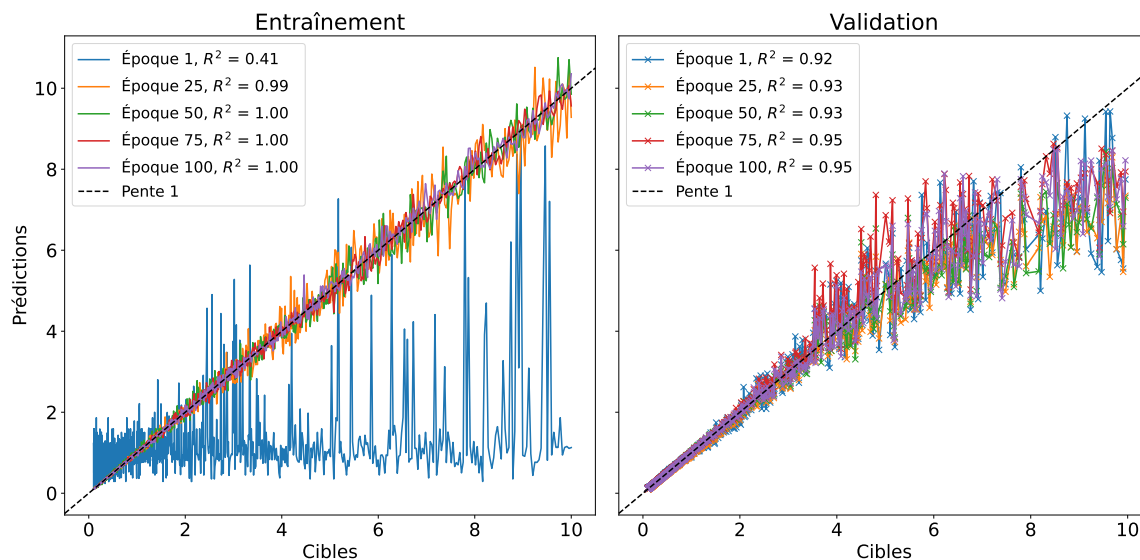


FIGURE 8.19 – Prédictions en fonction des cibles du le cinquième réseau pour l’entraînement et la validation. On affiche quelques époques par graphique.

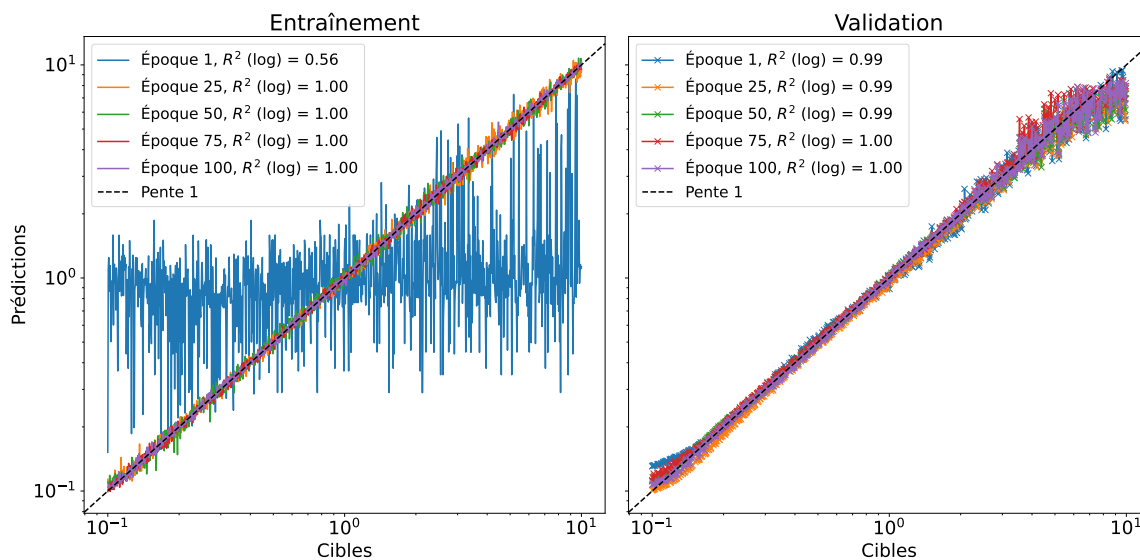


FIGURE 8.20 – Log-prédictions en fonction des log-cibles du le cinquième réseau pour l’entraînement et la validation. On affiche quelques époques par graphique.

8.8 Analyses supplémentaires

Dans tous les cas étudiés jusqu'à présent, on note une difficulté à traiter les cas où les cibles sont plus élevées que le temps d'intégration. Deux explications indépendantes peuvent être expliquées. Premièrement, en générant les temps caractéristiques sur une échelle logarithmique, on met l'emphasis sur les petites valeurs, obtenant plus de τ_c dans les régimes $\tau_c < T$ et $\tau_c \approx T$. Cela peut créer un déséquilibre dans les données, donc un apprentissage moins efficace dans les cas éloignés de $\tau_c > T$. Le choix de l'échelle logarithmique n'est pas anodin. Dans la physique des tavelures, on retrouve plus souvent des cas $\tau_c < T$, car la décorrélation des matériaux se fait plus rapidement que l'intégration ou l'accumulation des pixels dans une caméra. Plus τ_c dépasse T , plus c'est un régime biologique rare, sauf si on s'intéresse à des surfaces rigides. Dans le cas biomédical, on reste souvent dans le cas $\tau_c < T$ ou $\tau_c \approx T$.

Deuxièmement, les réseaux utilisent des couches de convolutions 2D qui déterminent les caractéristiques importantes pour la partie récurrente. Ces convolutions n'ont accès qu'aux pixels d'une même image, ils n'ont pas de références explicites au temps. On sait toutefois dans la physique des tavelures que le contraste spatial, soit $C = \frac{\sigma}{\mu}$ où σ est l'écart-type d'intensité et μ en est la moyenne, dépend implicitement des propriétés temporelles. Dans le cas d'une corrélation temporelle exponentielle, on a la formule[3]

$$C = \sqrt{\beta \frac{\tau_c}{2T} \left[2 + \frac{\tau_c}{T} \left(\exp\left\{ -\frac{2T}{\tau_c} \right\} - 1 \right) \right]}$$

où τ_c est le temps caractéristique, T est le temps d'intégration et β est un paramètre qu'on va supposer égal à 1. La figure 8.21 montre cette formule graphiquement, mais avec $T = 1$. Plus τ_c

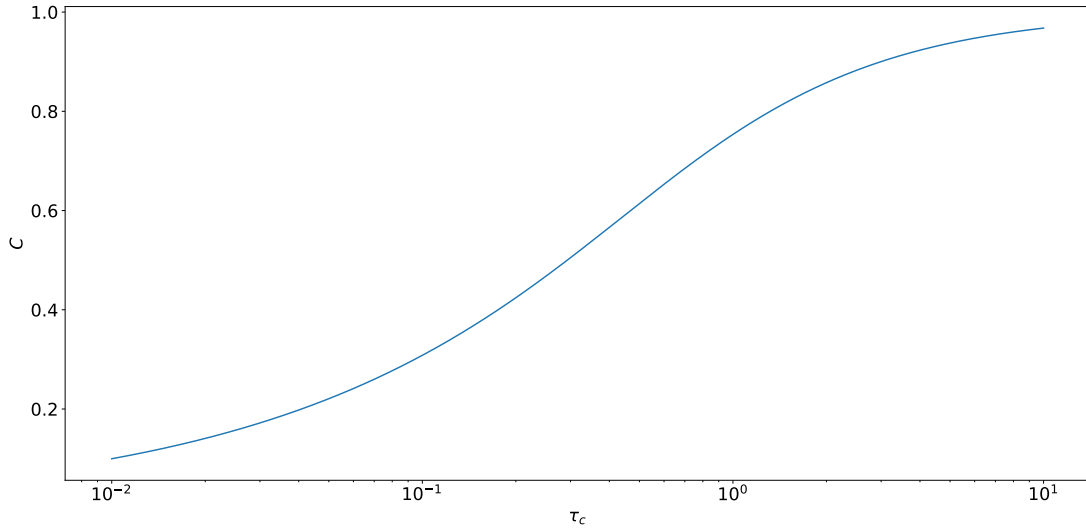


FIGURE 8.21 – Contraste spatial en fonction du temps caractéristique τ_c pour un temps d'intégration constant à 1.

augmente, plus le contraste approche 1 en asymptote, donc peu importe τ_c , le contraste reste proche de 1 et ne donne plus beaucoup d'information à propos de la corrélation temporelle. Dans le régime $\tau_c \approx T$, on a un contraste croissant rapidement en fonction de τ_c , donc beaucoup d'information à propos de la corrélation. Finalement, dans le cas $\tau_c < T$, on a aussi une asymptote à 0, donc si le temps caractéristique devient beaucoup plus petit que T , le contraste, un fois de plus, n'apporte pas d'information sur la corrélation temporelle. C'est ce qui pourrait expliquer les petites divergences dans les figures prédictions en fonction des cibles en échelle logarithme pour les petits τ_c . La figure 8.22, tirée de [3], montre graphiquement que lorsque le ratio T/τ_c change, la différence de contraste devient plus ou moins évidente, donc renferme plus ou moins d'information. On voit que la différence de contraste est maximale lorsque T est proche des temps caractéristiques impliqués.

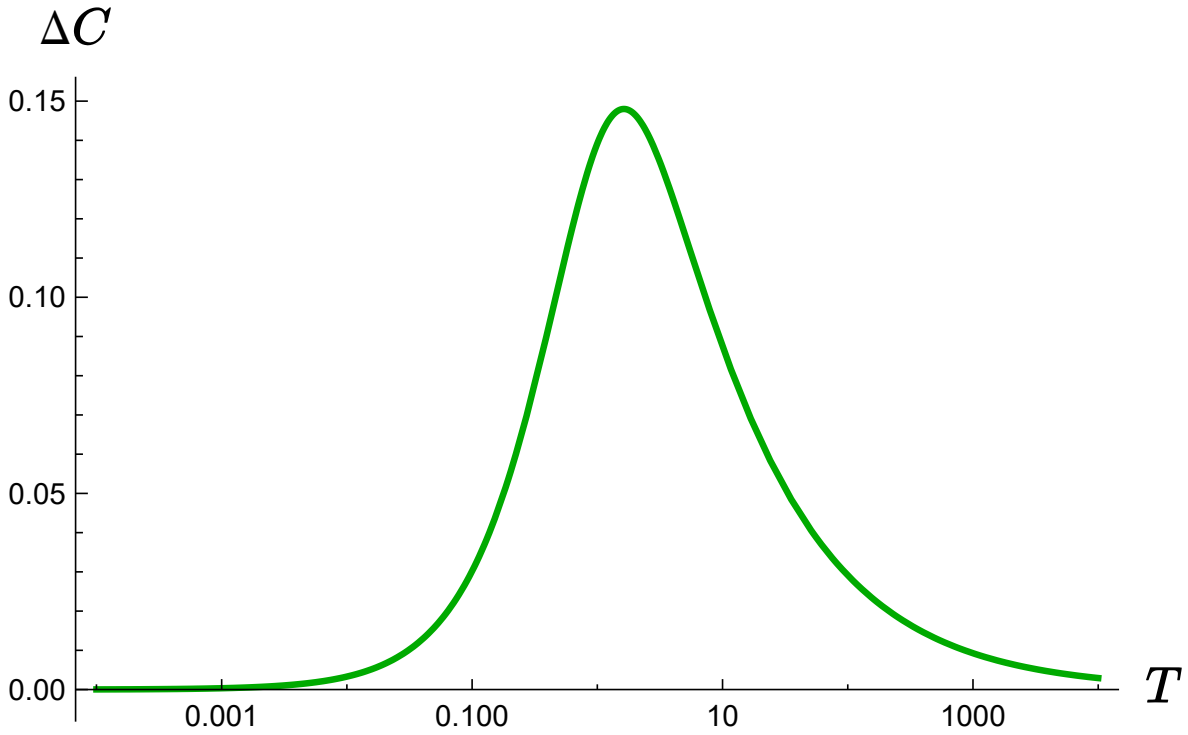


FIGURE 8.22 – Différence de contraste spatial en fonction du temps d'acquisition entre des tavelures corrélées exponentiellement avec $\tau_c = 0.5$ et $\tau_c = 1$.

Chapitre 9

Seconde partie du projet

Dans la seconde moitié de la session, il sera question d’optimiser le réseau. On sait qu’il est possible d’apprendre et de généraliser sous certaines conditions. Il sera important de varier les conditions et d’augmenter la complexité. L’évaluation finale se fera ainsi à ce moment.

9.1 Optimisation

Dans le chapitre précédent, différentes architectures et différents hyperparamètres sont testés. Le dernier est le plus prometteur, mais il est peut-être possible d’en avoir un meilleur. Il peut être important d’effectuer une recherche d’hyperparamètres, comme le taux d’apprentissage, le nombre de couches de convolution, leurs tailles, leurs nombres de filtres, ainsi que la partie récurrente avec le nombre de couches GRU et le nombre d’unités cachées. De plus, il serait intéressant de tester l’ajout de composants qui sont normalement présent dans la plupart des architectures CNN, comme une normalisation entre les couches. Il ne sera pas question de *batch norm* ici, car la structure d’une batch peut contenir différentes séquences temporelles indépendantes et on mélangerait ainsi les statistiques, brouillant l’information. *Instance norm* est considéré. Dans la plupart des cas, ajouter une couche de normalisation aide la convergence. Finalement, éventuellement il serait important d’ajouter une couche de normalisation $[0, 1]$ en entrée du réseau, permettant de mettre toutes les images à la même échelle dans l’intervalle $[0, 1]$. Contrairement à une normalisation $z - score$, on ne modifie pas vraiment la nature des distributions de probabilité, on ne fait que les translater dans un certain intervalle.

9.2 Varier les conditions

On s’est pour l’instant seulement intéressé à une fonction de corrélation et un seul temps d’intégration lors des tests présentés dans le chapitre 8. Or, n’avoir qu’un seul temps d’intégration n’est pas critique, car c’est le ratio T/τ_c qui joue un rôle primordial dans les statistiques spatiales, comme le contraste. En faisant varier τ_c seulement, on fait varier ce ratio sans avoir de

temps d'intégration qui varie. Il serait toutefois intéressant de tester si le réseau entraîné sur, par exemple $T = 1$ performe aussi bien sur des données avec $T = 2$ et des ratios identiques. Ajouter différentes fonctions de corrélation est intéressant, car on peut vérifier la robustesse du réseau à prédire le temps de corrélation même si elle est structurellement différente. Cela n'est point important vers l'obtention d'un modèle d'analyse demandant le moins de paramètres possibles, comme la fonction de corrélation qui est plus souvent qu'autrement inconnue.

Finalement, on peut aussi penser à ajouter un masque gaussien qui vient atténuer l'intensité plus on s'éloigne du centre de l'image. Cela permet de simuler des images plus réalistes où l'illumination n'est pas uniforme sur toute l'image, par exemple un laser qui illumine seulement une tache à profil gaussien.

9.3 Sélection d'un modèle scalaire final

On a effectué une certaine analyse d'apprentissage et de généralisation dans le chapitre 8. Il est important de bâtir sur cette analyse, notamment en testant la cohérence de prédiction. Si on donne en test plusieurs séquences ayant la même cible, il est important de déterminer si les prédictions sont similaires. La cohérence d'un réseau est primordiale, sinon il ne vaut rien.

9.4 Modèle cartographique : un aperçu

Si le temps le permet, il serait intéressant d'évaluer la pertinence et la faisabilité d'un réseau cartographique. Au lieu de donner un scalaire comme sortie (ce qui est le cas dans la première partie), le réseau prédirait une carte locale de temps caractéristique. Cette approche est très intéressante d'un point de vue biomédical, car on pourrait extraire de l'information locale et ensuite utiliser d'autres outils d'analyse pour faire une tâche quelconque. Par exemple, on image le cortex cérébral et à partir de cette carte locale, on peut distinguer les vaisseaux sanguins du background par leur temps caractéristique normalement plus petit. Cette approche fait penser au *laser speckle contrast imaging* (LSCI) où on image des tavelures et ensuite on applique des transformations pour calculer le contraste et extraire de l'information spatiale de résolution intéressante. Le problème avec le LSCI est surtout au niveau de la paramétrisation qui est assez lourde et handicapante.

Bibliographie

- [1] David BRIERS, Donald D. DUNCAN, Evan HIRST, Sean J. KIRKPATRICK, Marcus LARSSON, Wiendelt STEENBERGEN, Tomas STROMBERG et Oliver B. THOMPSON : Laser speckle contrast imaging : theoretical and practical limitations. *Journal of Biomedical Optics*, 18(6) :066018, juin 2013.
- [2] Ingemar FREDRIKSSON, Martin HULTMAN, Tomas STRÖMBERG et Marcus LARSSON : Machine learning in multiexposure laser speckle contrast imaging can replace conventional laser Doppler flowmetry. *Journal of Biomedical Optics*, 24(01) :1, janvier 2019.
- [3] Gabriel GENEST : Exploration théorique et simulations des tavelures laser dans un contexte d'imagerie, 2025.
- [4] Joseph W. GOODMAN : *Statistical Optics*. Wiley Series in Pure and Applied Optics. John Wiley & Sons Inc, Hoboken, New Jersey, 2e édition, 2015.
- [5] Joseph W. GOODMAN : *Speckle Phenomena in Optics : Theory and Applications, Second Edition*. SPIE, janvier 2020.
- [6] Jie HU, Li SHEN, Samuel ALBANIE, Gang SUN et Enhua WU : Squeeze-and-Excitation Networks, 2017.
- [7] Martin HULTMAN, Marcus LARSSON, Tomas STRÖMBERG et Ingemar FREDRIKSSON : Speed-resolved perfusion imaging using multi-exposure laser speckle contrast imaging and machine learning. *Journal of Biomedical Optics*, 28(03), mars 2023.
- [8] Edward JAMES, Samuel POWELL et Peter MUNRO : Simulation of statistically accurate time-integrated dynamic speckle patterns in biomedical optics. *Optics Letters*, 46(17) : 4390, septembre 2021.
- [9] Nikhil KALER, Vimal BHATIA et Amit Kumar MISHRA : Deep Learning-Based Robust Analysis of Laser Bio-Speckle Data for Detection of Fungal-Infected Soybean Seeds. *IEEE Access*, 11 :89331–89348, 2023.
- [10] Diederik P. KINGMA et Jimmy BA : Adam : A Method for Stochastic Optimization, 2014.

- [11] Min LIN, Qiang CHEN et Shuicheng YAN : Network In Network, mars 2014. arXiv :1312.4400.
- [12] Chang LIU, Kivilem KILIÇ, Sefik Evren ERDENER, David A. BOAS et Dmitry D. POSTNOV : Choosing a model for laser speckle contrast imaging. *Biomedical Optics Express*, 12(6) :3571, juin 2021.
- [13] Kai Jing SHANG, Yuan YUAN, Hong Li LIU, Ren Bing WANG, Wei Nan GAO, Yong BI et Yang YU : Estimation of absolute wide-range blood flow by deep learning-based laser speckle contrast imaging. *Optics and Lasers in Engineering*, 193 :109056, octobre 2025.
- [14] Shuqi ZHENG et Jerome MERTZ : Correcting sampling bias in speckle contrast imaging. *Optics Letters*, 47(24) :6333, décembre 2022.
- [15] Shuqi ZHENG et Jerome MERTZ : Direct characterization of tissue dynamics with laser speckle contrast imaging. *Biomedical Optics Express*, 13(8) :4118, août 2022.